

Bootcamp Inteligencia Artificial

Nivel Innovador

TALENTO
TECH

Master Class 2:

Deep Learning

Agenda

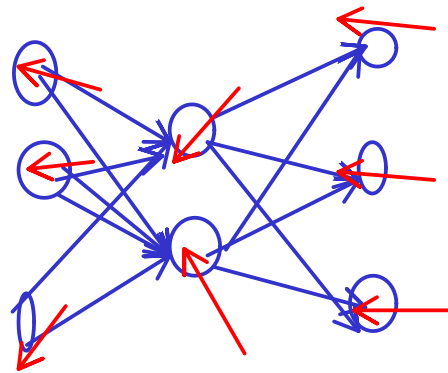
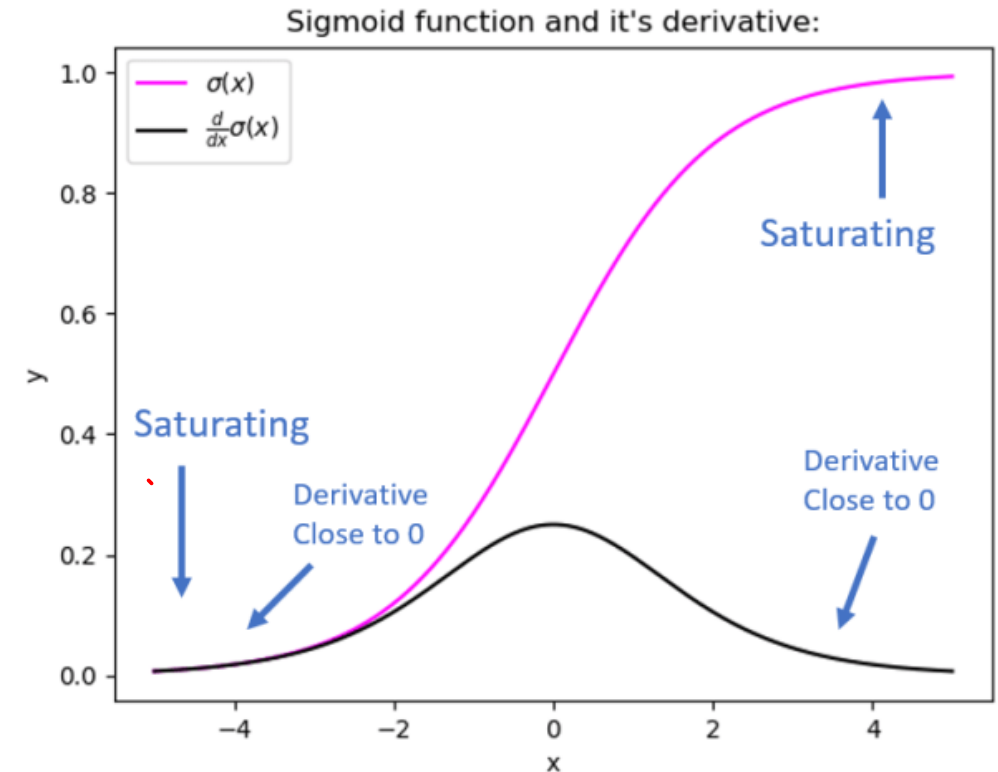
1. Entrenando Redes Profundas
2. Cargar y Preprocesar Datos con Tensorflow
3. Redes Neuronales Convolucionales

1.1 Entrenando Redes Profundas

El entrenamiento de redes profundas es un componente esencial en el campo del aprendizaje profundo, que ha demostrado ser excepcionalmente efectivo en una variedad de tareas, desde la visión por computadora hasta el procesamiento de lenguaje natural.

1.2 Vanishing Gradient y Exploding Gradient

Uno de los desafíos fundamentales en el entrenamiento de redes profundas es el problema de los gradientes que desaparecen o explotan durante la retropropagación. En capas profundas, especialmente con funciones de activación como la sigmoide o la tangente hiperbólica, los gradientes pueden volverse extremadamente pequeños (vanishing) o extremadamente grandes (exploding). Esto afecta la capacidad de las capas anteriores para aprender y ajustar sus pesos de manera efectiva.



Error

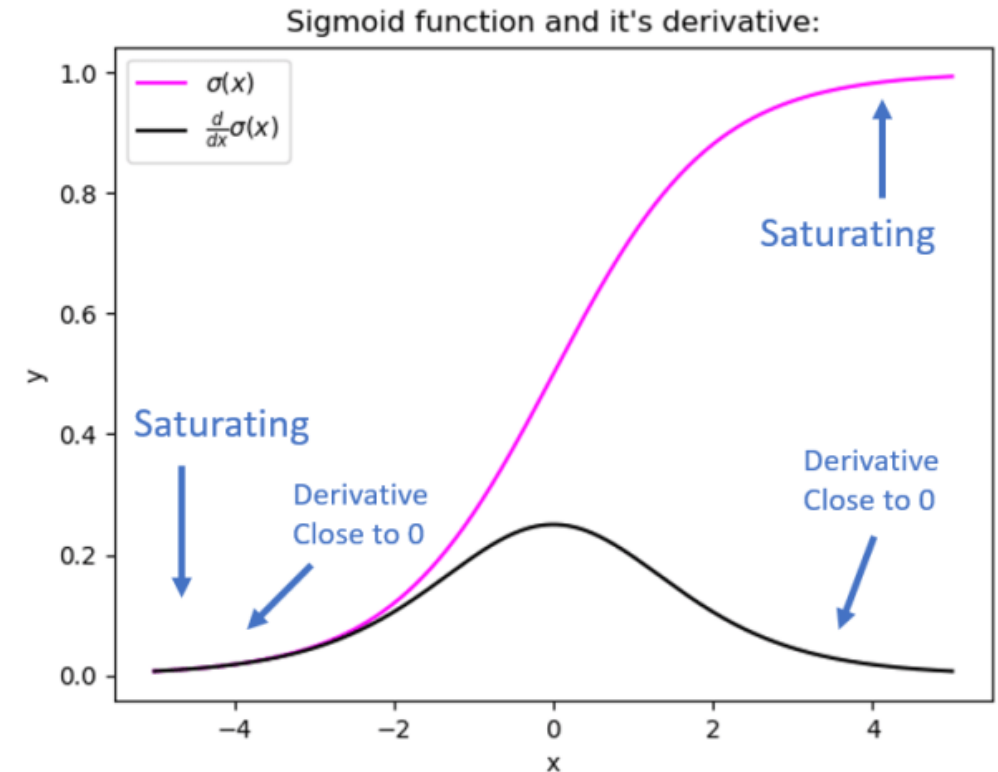
$$w(t+1) = w(t) + \eta E x_i$$

1.3 Vanishing Gradient y Exploding Gradient

Estrategias de Mitigación

- **Inicialización de Pesos:**

Inicializar los pesos de manera cuidadosa puede mitigar el problema de los gradientes vanishing/exploding. Técnicas como la inicialización de Xavier/Glorot y He consideran la escala de los pesos y ayudan a mantener una propagación adecuada de los gradientes.

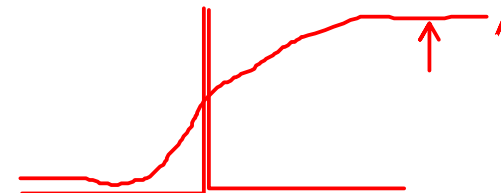
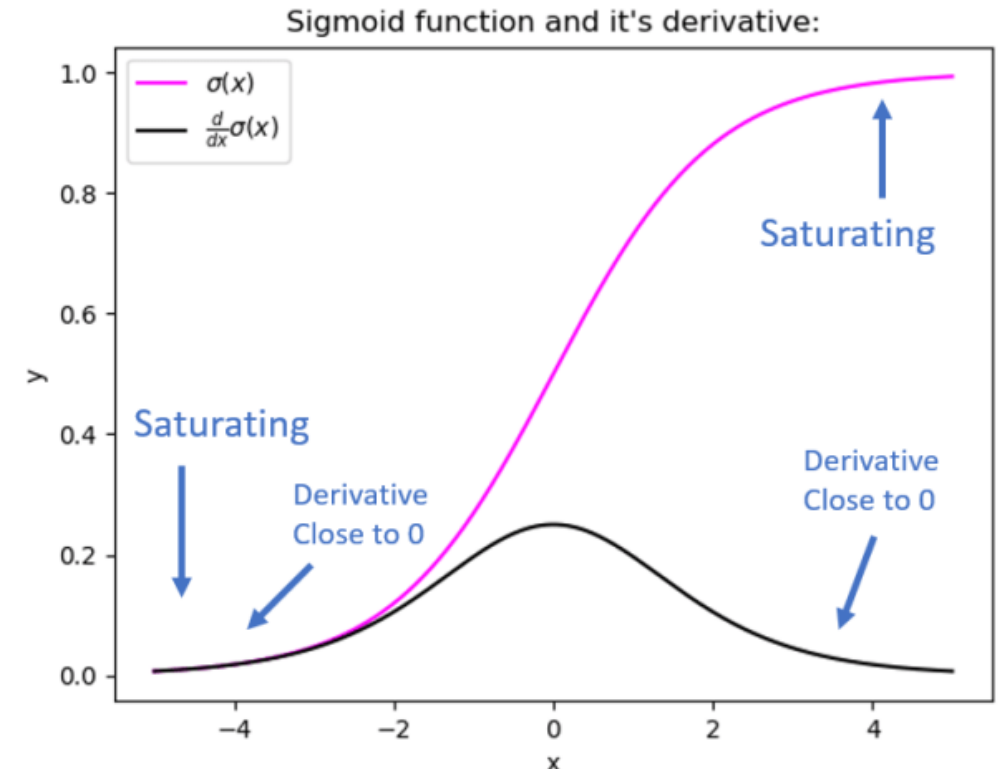
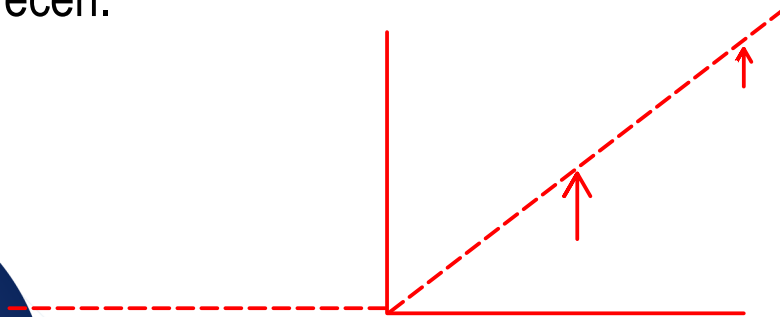


1.4 Vanishing Gradient y Exploding Gradient

Estrategias de Mitigación

- Uso de Funciones de Activación Adecuadas:

La elección de funciones de activación impacta directamente en el comportamiento de los gradientes. ReLU, al ser no saturante para activaciones positivas, ha demostrado mitigar el problema de los gradientes que desaparecen.

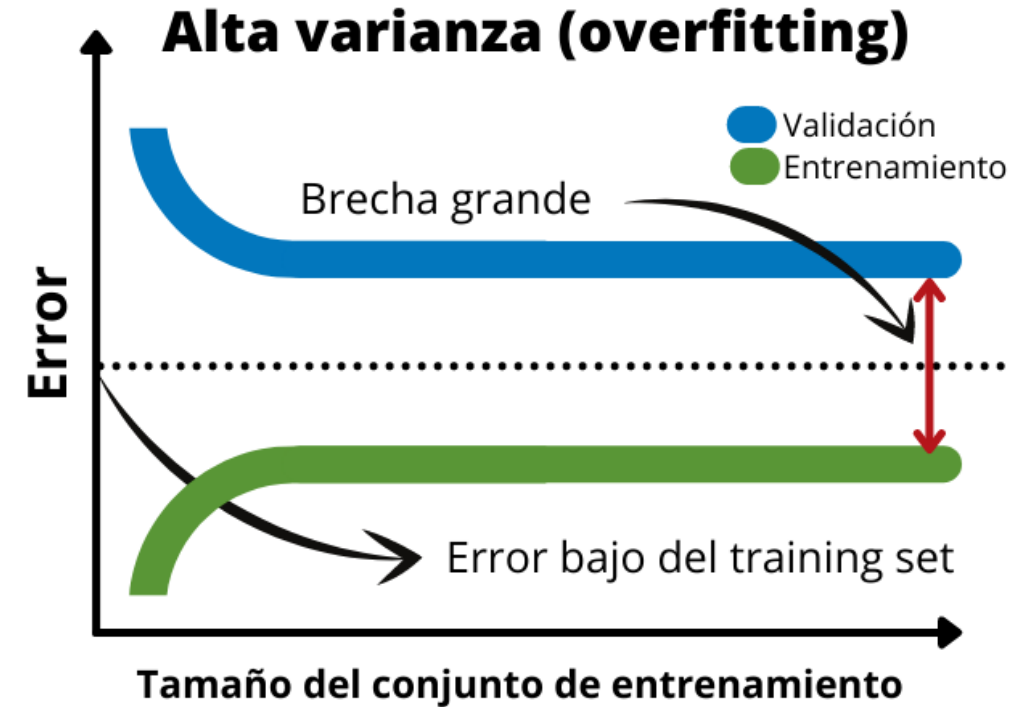


1. Partir los datos:: Training, testing
2. USo de los batch (lotes)

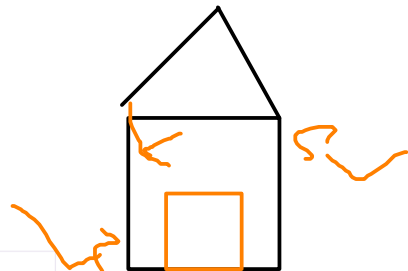
1.5 Overfitting en Redes Profundas

Con el aumento de la profundidad de la red, existe el riesgo de que el modelo se adapte demasiado a los datos de entrenamiento, lo que lleva a un fenómeno conocido como overfitting. En lugar de generalizar bien a nuevos datos, el modelo memoriza el conjunto de entrenamiento, lo que reduce su capacidad para realizar predicciones precisas en situaciones del mundo real.

Sobreentrenamiento: Red se aprende los parametros de entrada de MEMORIA



Aprender patrones NO DATOS!



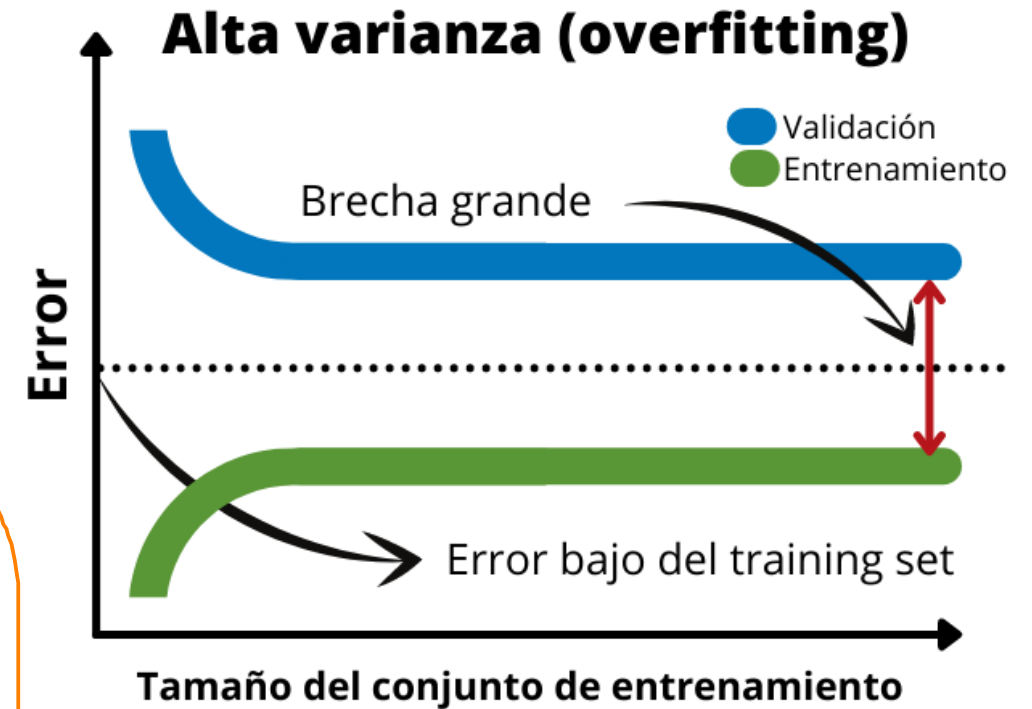
1.6 Overfitting en Redes Profundas

Promedio de los pesos
 $= 0$

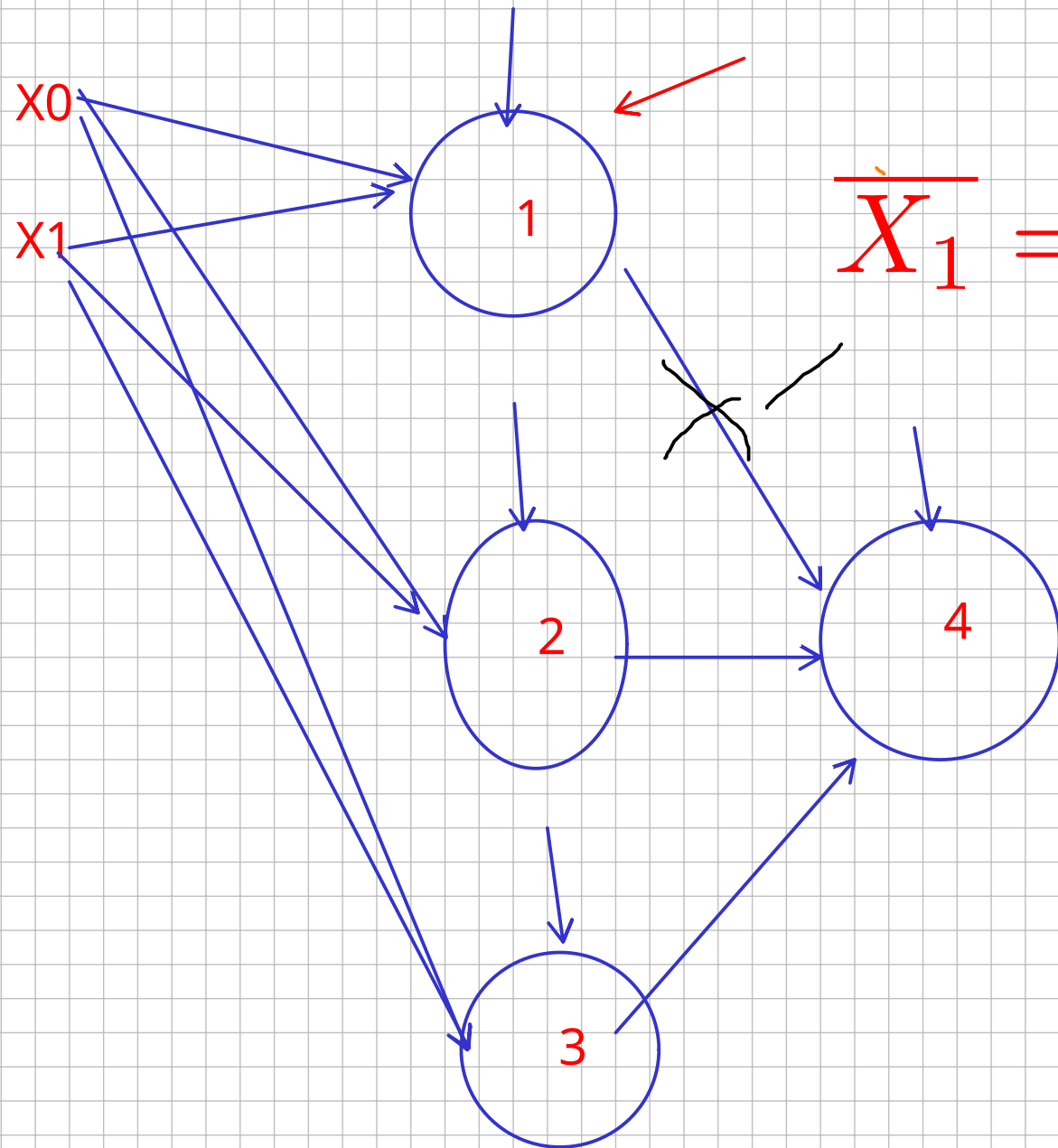
Estrategias de Mitigación:

Regularización: Técnicas como la regularización L1 y L2 pueden ser empleadas para penalizar pesos grandes y fomentar la simplicidad del modelo.

Dropout: Introducir dropout durante el entrenamiento implica aleatoriamente desactivar un porcentaje de neuronas en cada paso, lo que ayuda a prevenir la dependencia excesiva entre unidades y, por lo tanto, evita el sobreajuste.



Pesos desviación estandar
pequeña



$$\overline{X_1} = \sum_{i=0}^n w_{1,i} x_i + b_1$$

$$X_1 = \cancel{w_{11} x_1} + \underline{w_{12} x_2 + b_1}$$

f(X1)

$$w_{11} = 1000$$

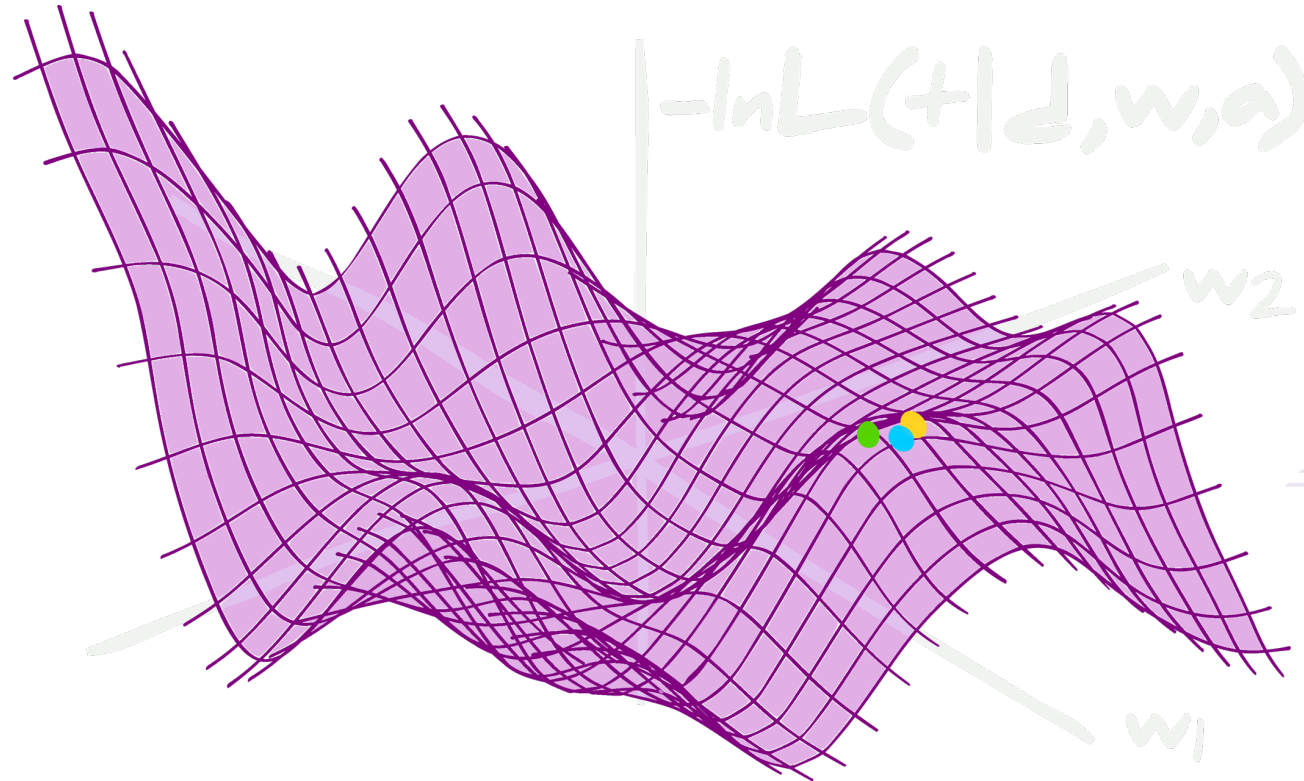
$$w_{12} = 0.5$$

$$b_1 = 0.5$$

$$1000 * ? + 0.5 * ? + 0.5 * ? = ?$$

1.7 Gradient Descent Lento

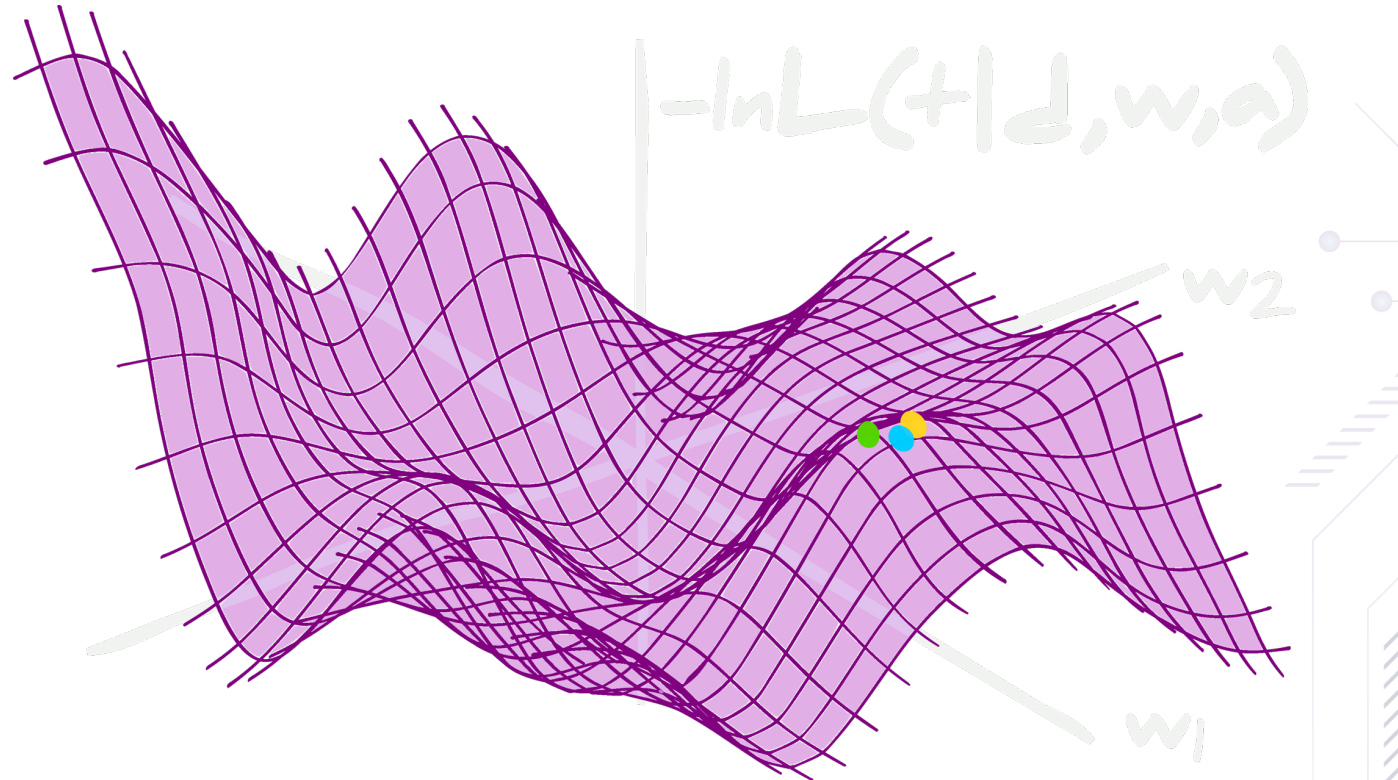
A medida que la red se vuelve más profunda, el descenso de gradiente básico puede volverse lento debido a la propagación de gradientes pequeños a través de muchas capas.



1.8 Gradient Descent Lento

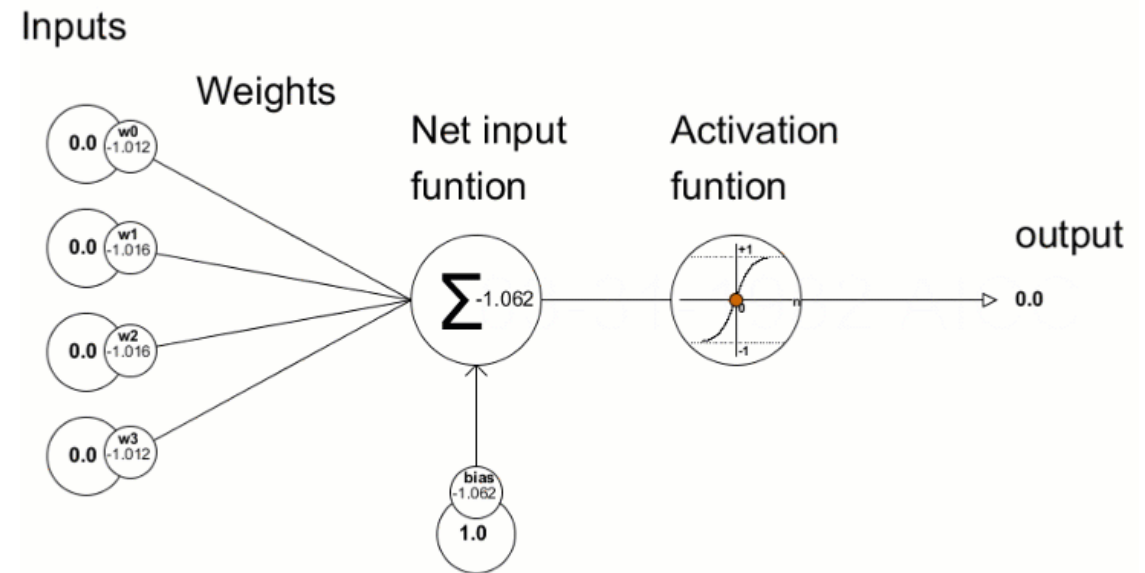
Estrategias de Mitigación:

Optimización Avanzada: Algoritmos de optimización avanzados, como Adam o RMSprop, adaptan automáticamente las tasas de aprendizaje para cada parámetro. Esto acelera la convergencia y ayuda a superar los problemas de gradientes lentos.



1.9 Inestabilidad Numérica

Operaciones numéricas en capas profundas pueden llevar a problemas de inestabilidad numérica, especialmente con funciones de activación como la tangente hiperbólica o la sigmoideal.



python

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> b = 1500
```

1500.0

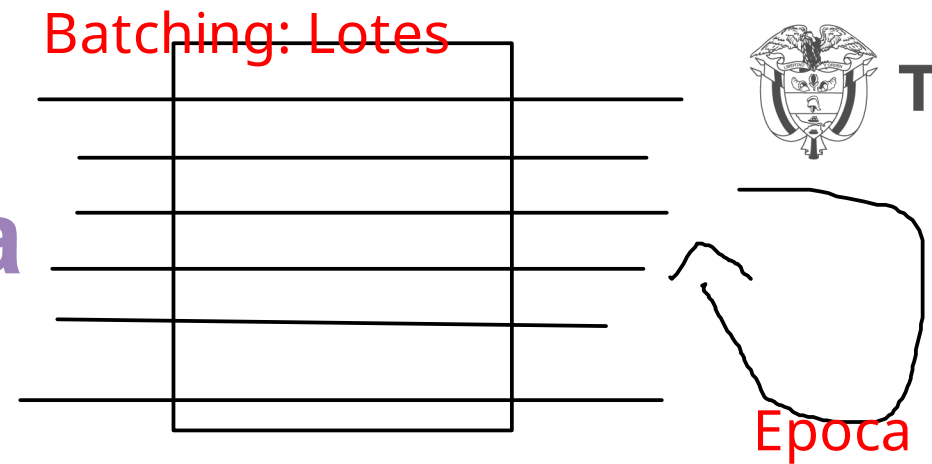
Time

ERROR C

10. *Journal of the American Medical Association*, 2000; 284: 2689-2694.

Es un valor tan pequeño que el sistema lo asume como 0, es decir si hay gradiente PERO NO es posible aplicarlo.

1.10 Inestabilidad Numérica

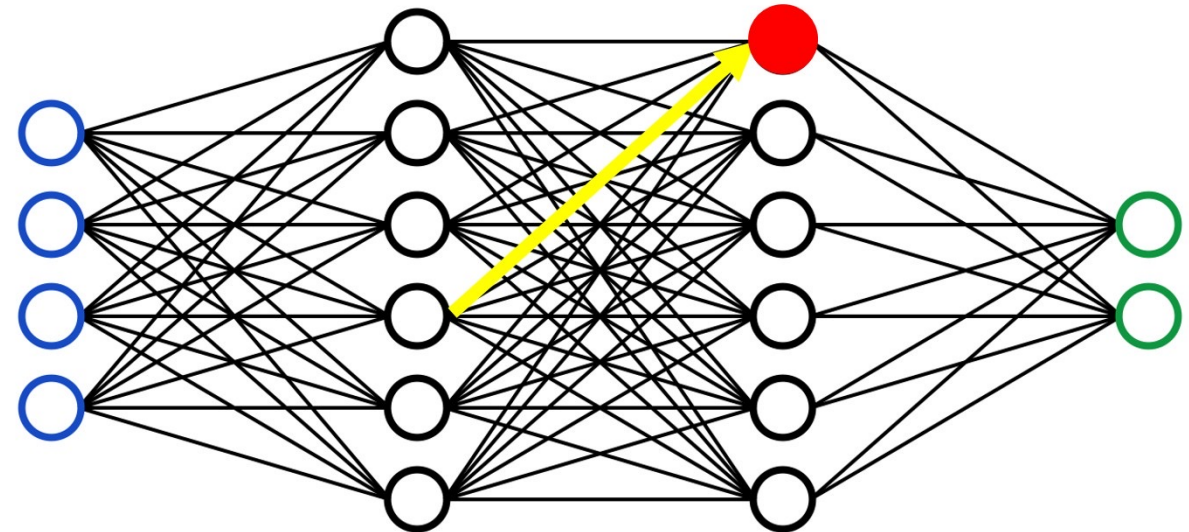


Batch normalization

Estrategias de Mitigación:

Normalización por Lotes: Batch Normalization normaliza las entradas de cada capa, reduciendo así el riesgo de inestabilidad numérica y acelerando la convergencia.

Las entradas se ajustan a la función de activación

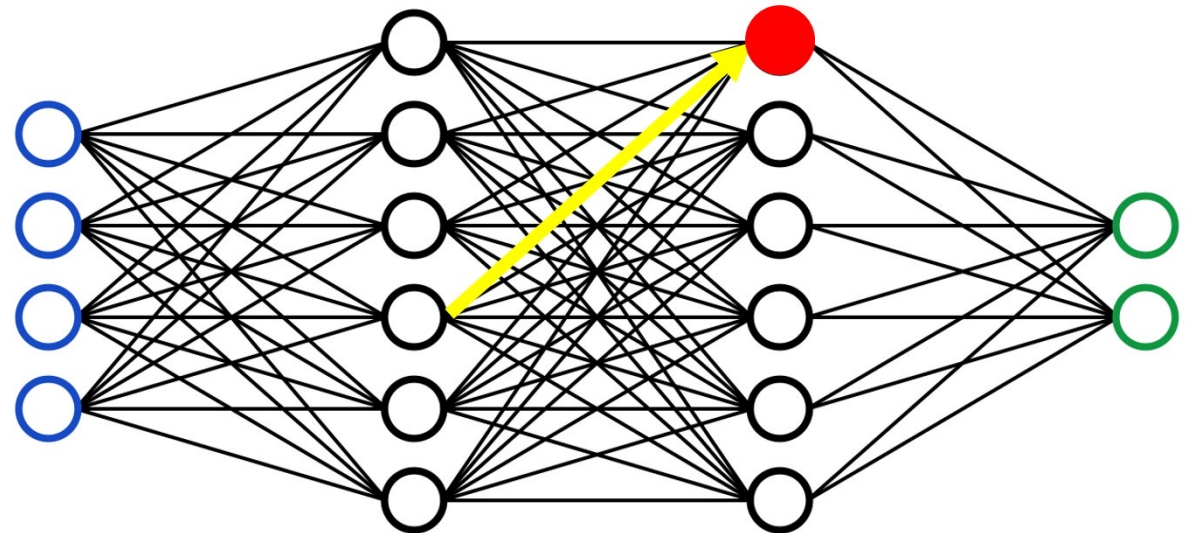


1.11 Inestabilidad Numérica

Estrategias de Mitigación:

Inicialización Cuidadosa: Inicializar los pesos de manera cuidadosa también puede ayudar a evitar problemas numéricos al mantener los valores en rangos manejables.

Batch normalization



1.12 Selección Adecuada de Hiperparámetros

Todo lo que se pueda configurar en la red neuronal

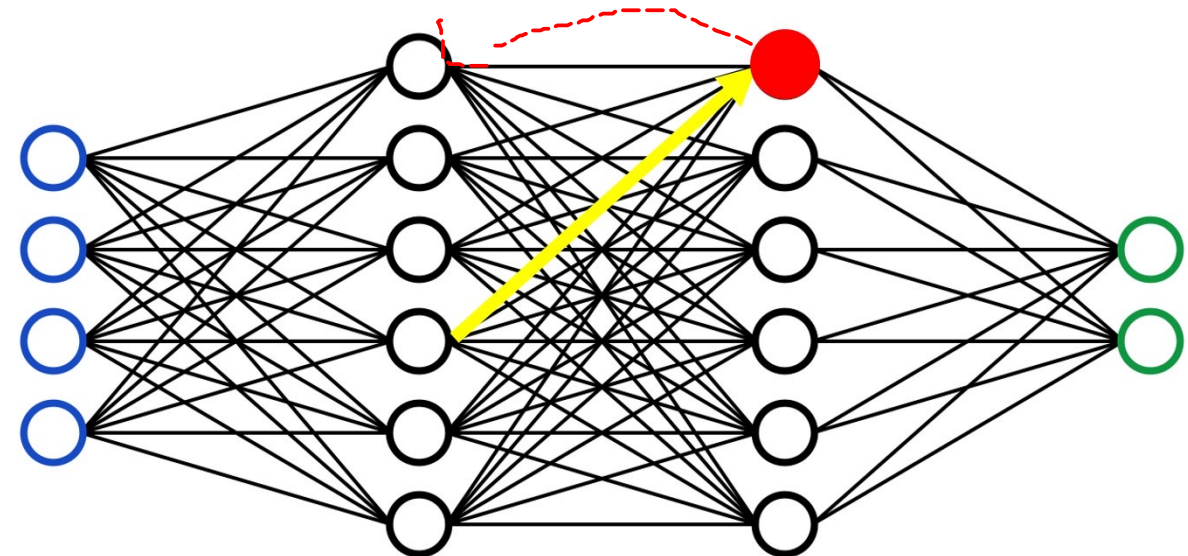
El ajuste adecuado de los hiperparámetros, como la tasa de aprendizaje y el tamaño del lote, es esencial para un entrenamiento efectivo.

Estrategias de Mitigación:

Búsqueda de Cuadrícula y : Explorar sistemáticamente combinaciones de hiperparámetros mediante búsqueda de cuadrícula o métodos más avanzados como la optimización bayesiana puede ayudar a encontrar configuraciones óptimas.

Ajuste Manual Iterativo: Ajustar hiperparámetros de forma iterativa, observando el rendimiento en el conjunto de validación, es una estrategia práctica para encontrar valores óptimos.

Batch normalization



1.13 Estrategias para el Entrenamiento Efectivo

Totalmente conectada

El entrenamiento efectivo de redes profundas va más allá de la elección de arquitecturas y requiere estrategias específicas para garantizar la convergencia rápida, la generalización adecuada y la estabilidad numérica durante el proceso. Aquí, exploraremos detalladamente algunas de las estrategias fundamentales para garantizar el éxito en el entrenamiento de redes profundas:

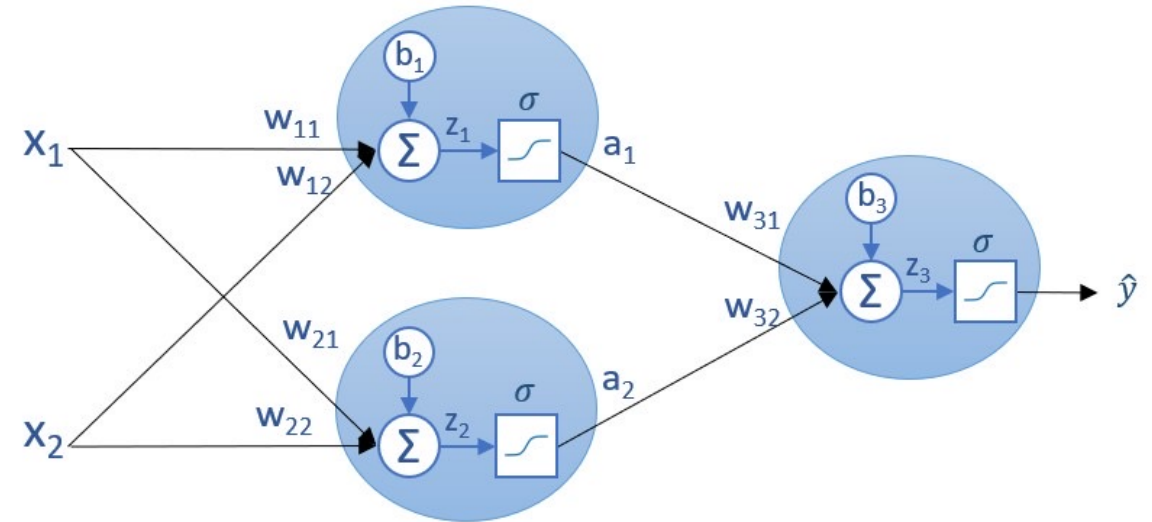
Patrones



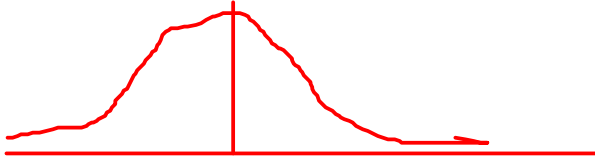
Gradientes (error)
no sean cero
ni muy grandes

1.14 Inicialización de Pesos

La inicialización de pesos desempeña un papel crucial en el entrenamiento de redes profundas. Inicializar los pesos de manera adecuada puede ayudar a mitigar problemas como los gradientes que desaparecen o explotan. Dos enfoques comunes son la inicialización de Xavier/Glorot y la inicialización de He.



1.15 Inicialización de Pesos



Estrategias de Inicialización:

Xavier/Glorot: Esta estrategia ajusta los pesos inicialmente asignados a las neuronas, teniendo en cuenta el número de entradas y salidas de la capa. Funciona bien con funciones de activación lineales como ReLU.

```
1 # Ejemplo de inicialización de pesos con Keras
2 from keras.initializers import GlorotNormal, HeNormal
3
4 # Xavier/Glorot
5 model.add(Dense(128, activation='relu', kernel_initializer=GlorotNormal()))
6
7 # He
8 model.add(Dense(128, activation='relu', kernel_initializer=HeNormal()))
```

1.16 Inicialización de Pesos

Normal: Mayor probabilidad
cerca del valor promedio
Uniform: Misma probabilidad

Estrategias de Inicialización:

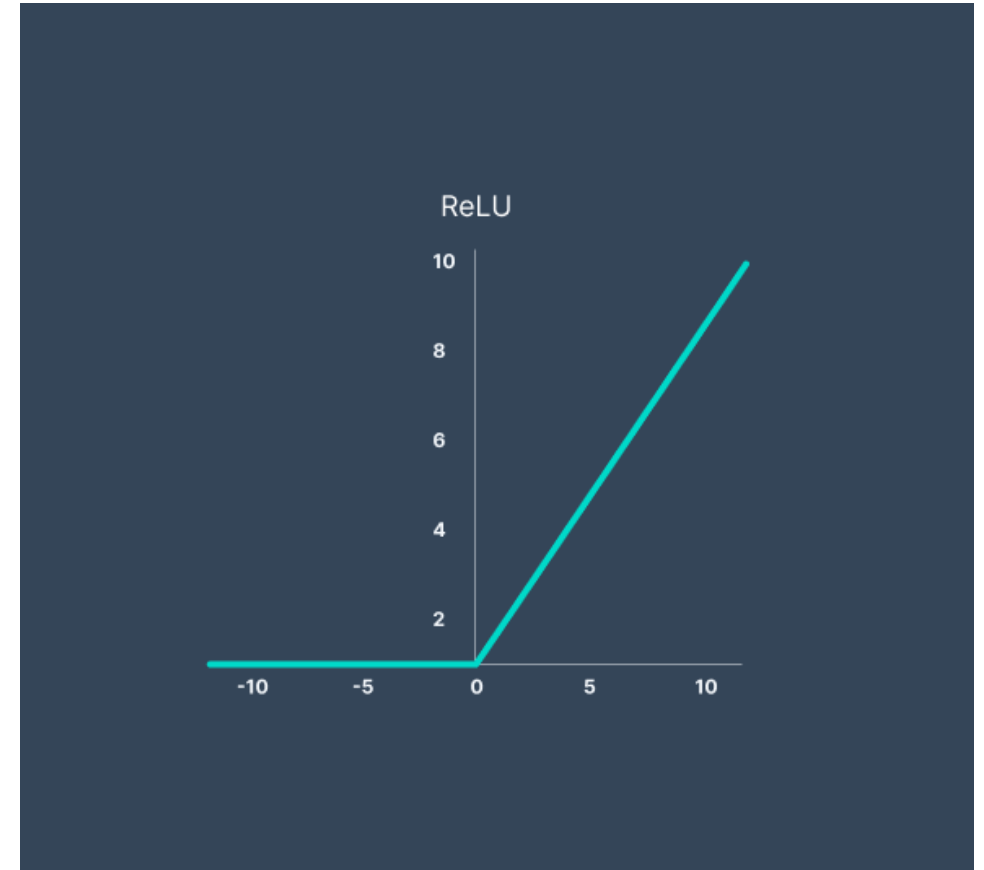
He: Similar a Xavier, pero ajusta los pesos de manera diferente para activaciones ReLU. Es especialmente útil cuando se utiliza ReLU como función de activación. desempeña un papel crucial en el entrenamiento de redes profundas. Inicializar los pesos de manera adecuada puede ayudar a mitigar problemas como los gradientes que desaparecen o explotan. Dos enfoques comunes son la inicialización de Xavier/Glorot y la inicialización de He.

```
1 # Ejemplo de inicialización de pesos con Keras
2 from keras.initializers import GlorotNormal, HeNormal
3
4 # Xavier/Glorot
5 model.add(Dense(128, activation='relu', kernel_initializer=GlorotNormal()))
6
7 # He
8 model.add(Dense(128, activation='relu', kernel_initializer=HeNormal()))
```

1.17 Funciones de Activación

Funciones de Activación

La elección de funciones de activación afecta directamente el comportamiento de la red durante el entrenamiento. La función ReLU es comúnmente utilizada debido a su simplicidad y efectividad, pero existen variantes que abordan sus limitaciones.



1.18 Funciones de Activación

Funciones de Activación Comunes:

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$

Leaky ReLU: Introduce una pequeña pendiente para los valores negativos para evitar "neuronas muertas".

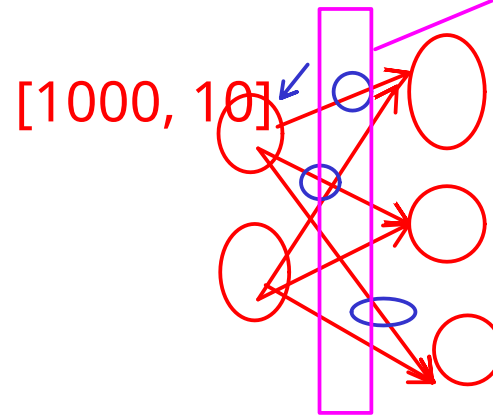
Parametric ReLU: Ajusta la pendiente durante el entrenamiento.

```
1  # Ejemplo de uso de funciones de activación en Keras
2  from keras.layers import Activation
3
4  # ReLU
5  model.add(Dense(128))
6  model.add(Activation('relu'))
7
8  # Leaky ReLU
9  model.add(Dense(128))
10 model.add(Activation('leaky_relu'))
11
12 # Parametric ReLU
13 model.add(Dense(128))
14 model.add(Activation('prelu'))
```

1.19 Normalización por Lotes (Batch Normalization)

La **normalización por lotes** es una técnica que normaliza las entradas de cada capa, lo que acelera la convergencia y actúa como una forma de regularización. Esto es especialmente útil en redes profundas donde la propagación de gradientes puede volverse problemática.

```
tutorial.py
1  # Ejemplo de Batch Normalization en Keras
2  from keras.layers import BatchNormalization
3
4  model.add(Dense(128))
5  model.add(BatchNormalization())
6  model.add(Activation('relu'))
```



[0.9, 0.05]

1.20 Optimización Avanzada

Los algoritmos de optimización son esenciales para ajustar los pesos de la red durante el entrenamiento. Algoritmos como Adam y RMSprop son populares debido a su capacidad para adaptar la tasa de aprendizaje de manera dinámica.

```
1  # Ejemplo de uso de optimizadores en Keras
2  from keras.optimizers import Adam
3
4  optimizer = Adam(learning_rate=0.001)
5  model.compile(optimizer=optimizer,
6               loss='categorical_crossentropy', metrics=['accuracy'])
```

1.21 Ajuste de Tasa de Aprendizaje

El ajuste adecuado de la tasa de aprendizaje es crucial para garantizar la convergencia del modelo. Puede ser beneficioso reducir la tasa de aprendizaje durante el entrenamiento para mejorar la precisión y estabilidad.

```
1  # Ejemplo de reducción de la tasa de aprendizaje en Keras
2  from keras.callbacks import ReduceLRonPlateau
3
4  reduce_lr = ReduceLRonPlateau(monitor='val_loss',
5                                factor=0.2, patience=5, min_lr=0.0001)
6  model.fit(x_train, y_train, validation_data=(x_val, y_val),
7           callbacks=[reduce_lr])
```

1.22 Estrategias para el Entrenamiento Efectivo

Estas estrategias se combinan para proporcionar un entrenamiento más estable y eficiente de las redes profundas. La elección y la configuración adecuadas de estas técnicas dependerán del problema específico y de las características del conjunto de datos. Experimentar con diversas configuraciones y monitorear el rendimiento es esencial para lograr un entrenamiento exitoso.

- Inicialización de Pesos He/Glorot
- Funciones de Activación RELU
- Normalización por Lotes (Batch Normalization) Normalización
- Optimización Avanzada ADAM/Rmsprop
- Ajuste de Tasa de Aprendizaje Ajuste del factor aprendizaje

Problema 1. Gradientes pueden ser cero o muy grandes, los pesos no cambian o se hacen muy grandes

Problema 2: Estabilidad numérica: Valores tan pequeños que se asumen con cero

1.23 Estrategias Específicas para Tareas

Las estrategias específicas para tareas en el entrenamiento de redes profundas se centran en aprovechar enfoques especializados para abordar problemas particulares. Estas técnicas van más allá de la simple arquitectura de la red y se adaptan a las características únicas de diferentes tipos de datos y tareas.

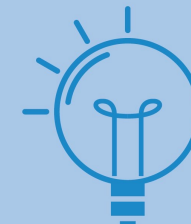
ARTIFICIAL INTELLIGENCE

Artificial Intelligence captures the imagination of the world.



MACHINE LEARNING

Machine learning starts to gain traction.



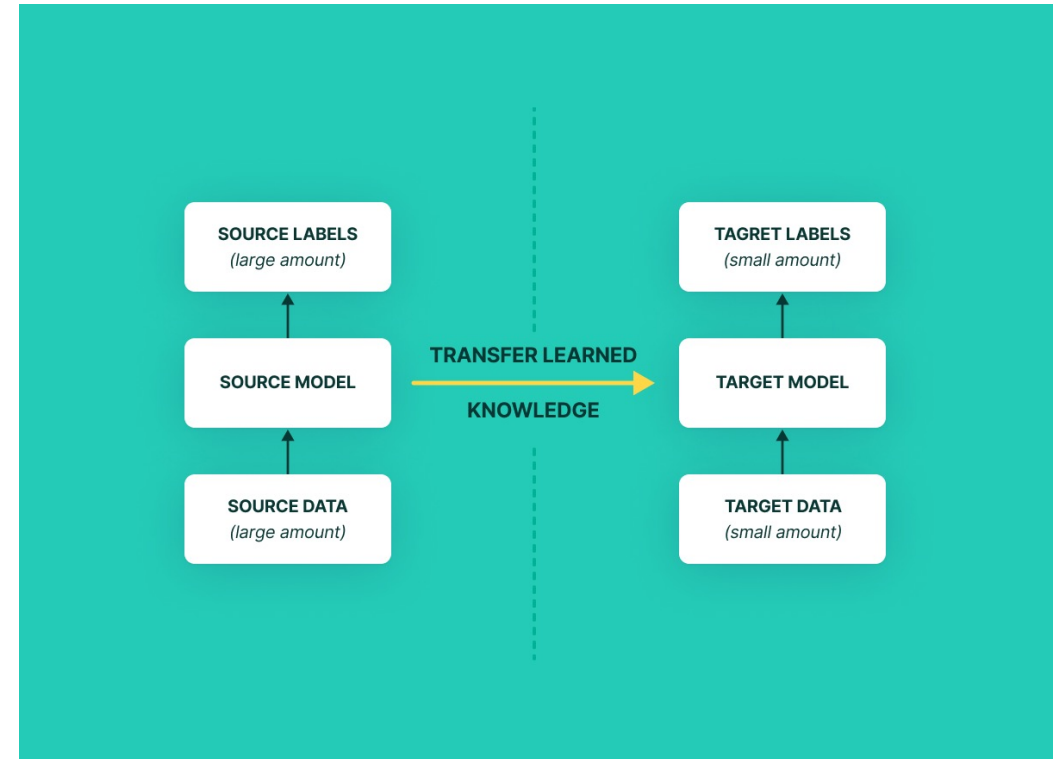
DEEP LEARNING

Deep learning catapults the industry.



1.24 Transfer Learning

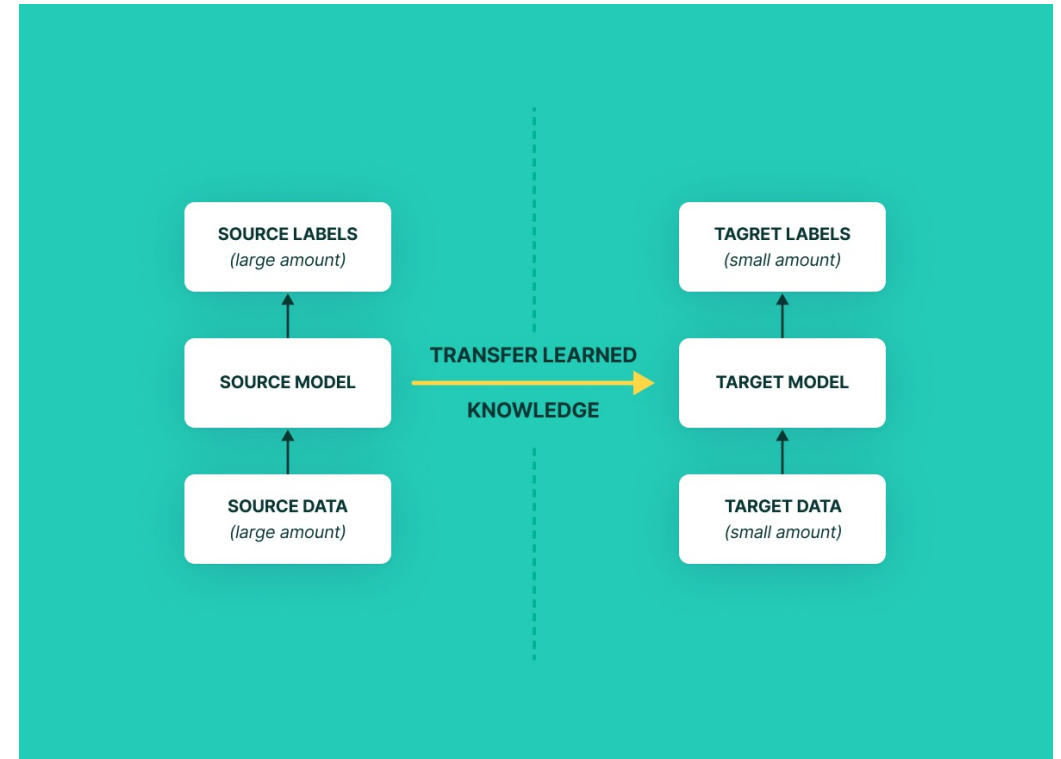
El Transfer Learning es una estrategia poderosa que aprovecha el conocimiento adquirido por un modelo entrenado previamente en una tarea para mejorar el rendimiento en una tarea relacionada. Esto es especialmente útil cuando se tienen conjuntos de datos pequeños.



1.25 Transfer Learning

Implementación Práctica:

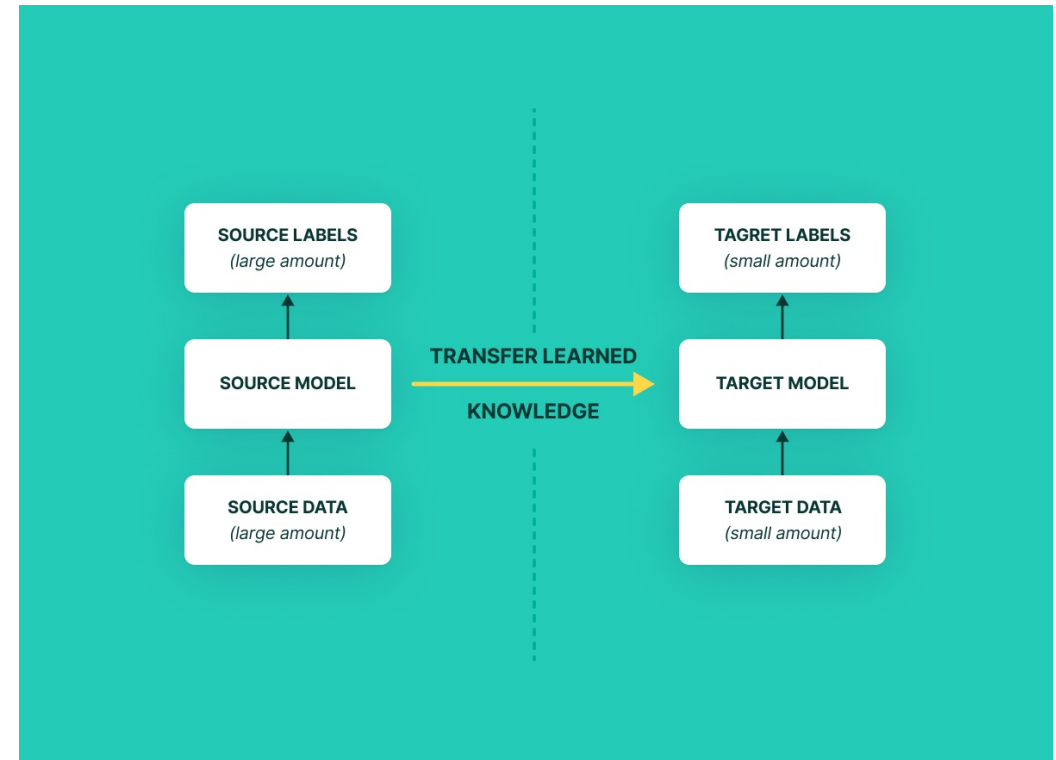
Fine-Tuning: Ajustar las capas superiores de un modelo preentrenado en una tarea específica. Esto permite que el modelo retenga conocimientos generales y se adapte a la nueva tarea.



1.26 Transfer Learning

Implementación Práctica:

Extracción de Características: Utilizar representaciones aprendidas por un modelo preentrenado como entradas para un nuevo modelo, a menudo añadiendo capas personalizadas en la parte superior para la tarea específica.

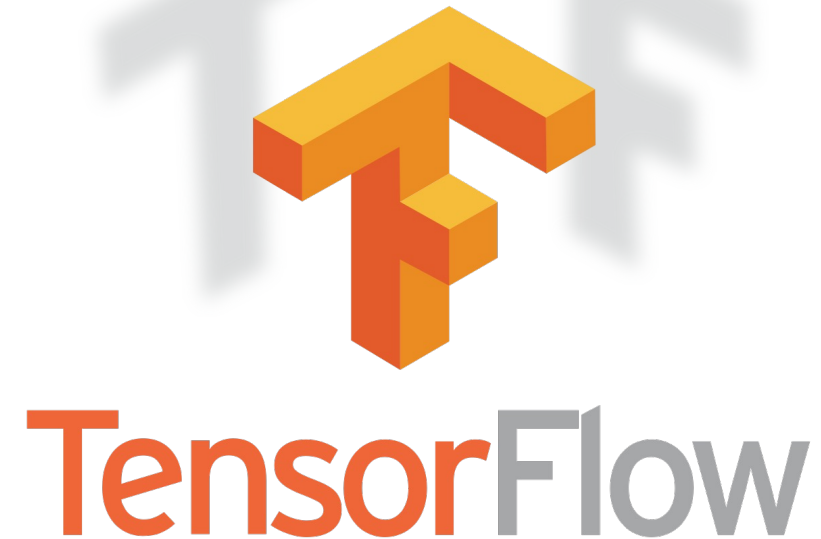


2.1 Introducción

Hasta ahora, solo hemos utilizado conjuntos de datos que caben en la memoria, pero los sistemas de aprendizaje profundo a menudo se entrenan en conjuntos de datos muy grandes que no caben en la RAM.

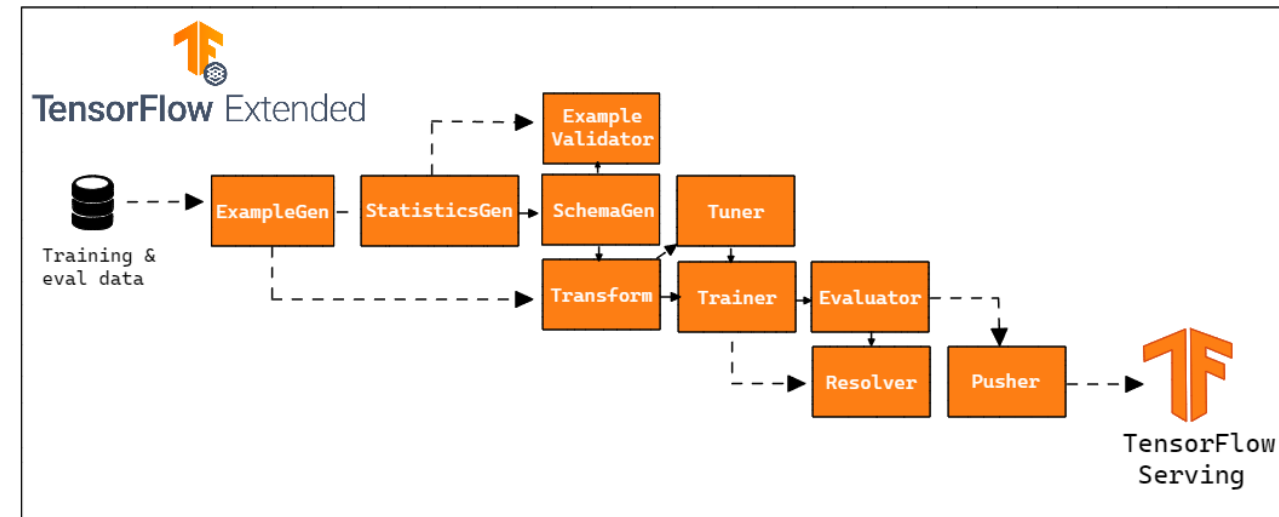
Ingerir un gran conjunto de datos y preprocesarlo de manera eficiente puede ser complicado de implementar con otras bibliotecas de aprendizaje profundo, pero TensorFlow lo hace fácil gracias a la API de datos:

simplemente crea un objeto de conjunto de datos, le indica dónde obtener los datos y luego lo transforma de la forma que desee, y TensorFlow se ocupa de todos los detalles de implementación, como procesamiento por lotes, captación previa, etc



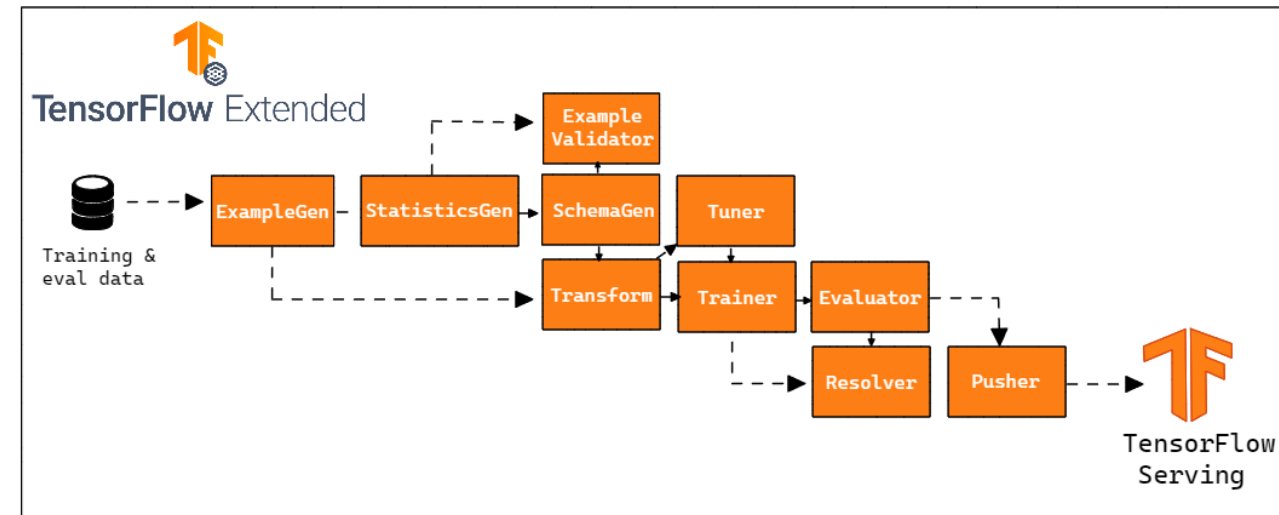
2.2 Pipeline de TensorFlow

El pipeline de TensorFlow ofrece diversos beneficios. Por ejemplo, supongamos que queremos hacer una aplicación de clasificación de comida, en donde tenemos que encontrar si una imagen aleatoria es un hot dog o no. Lo primero que necesitas es crear un set de datos de un montón de comida y etiquetarla con el nombre de lo que está en el plato.



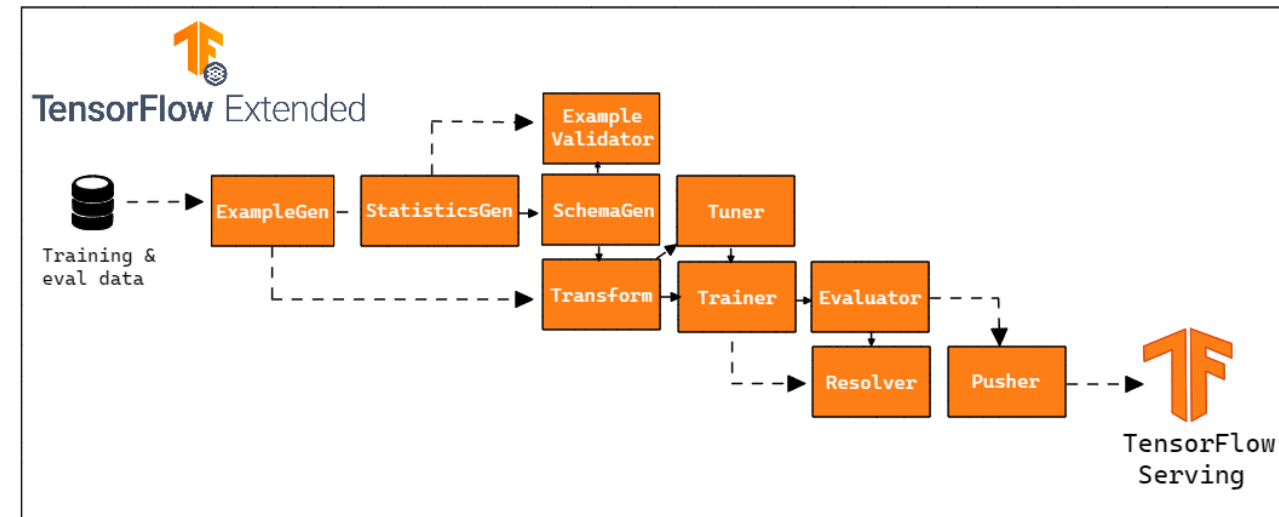
2.3 Pipeline de TensorFlow

Claramente al descargar estas imágenes, estás ahora vivirían en tu disco duro, sin embargo, al momento de ser utilizadas por el modelo, deben ser cargadas en RAM para que entren en algún arreglo de Numpy o un dataframe de Pandas. Estás imágenes deben convertirse en números porque los modelos de Machine Learning no pueden trabajar con la imagen como tal.



2.4 Pipeline de TensorFlow

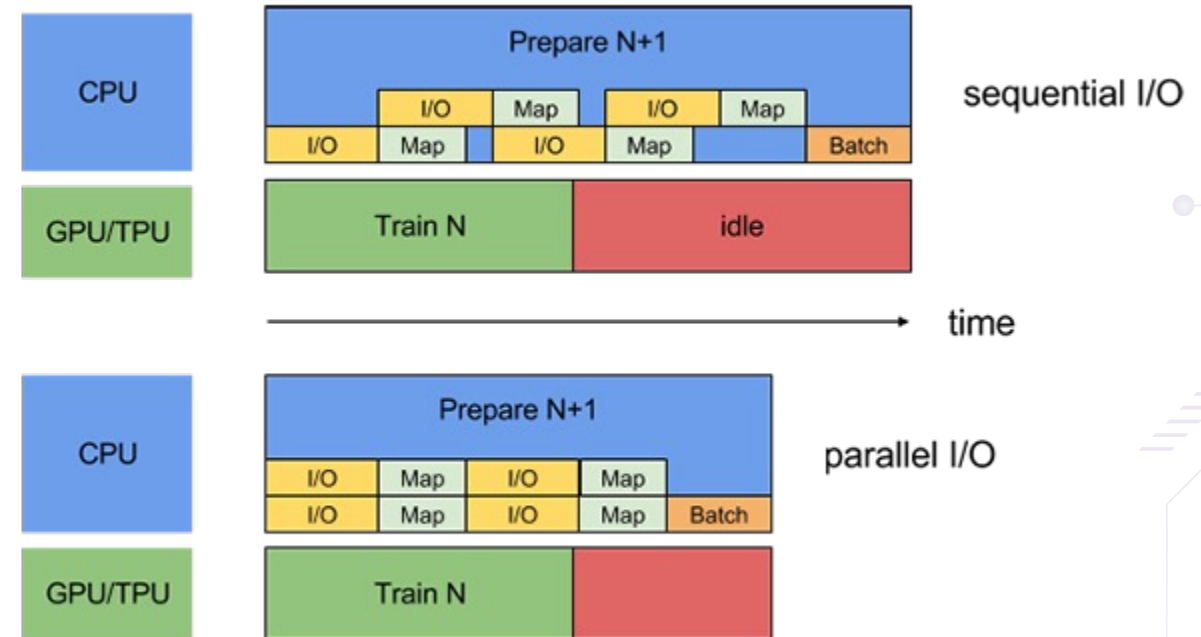
Sin embargo, esto se convierte en un problema por dos cuestiones, la primera es que necesitas una cantidad comprensible de imágenes, por ejemplo 10,000, para poder entrenar un modelo robusto que pueda clasificar correctamente la comida. La segunda es que existe un límite de RAM que no permite procesar 10,000 imágenes a la vez.



2.5 Pipeline de TensorFlow

Ante esto, se genera como alternativa crear mini batches. Es decir, en vez de cargar las 10,000 imágenes de una vez, se generan lotes (o batches) de 1,000 imágenes que pueden ser perfectamente procesables por la RAM. A este método se le conoce como *Streaming Approach* y para cargar estos lotes se necesita utilizar una herramienta de *TensorFlow* llamada datasets.

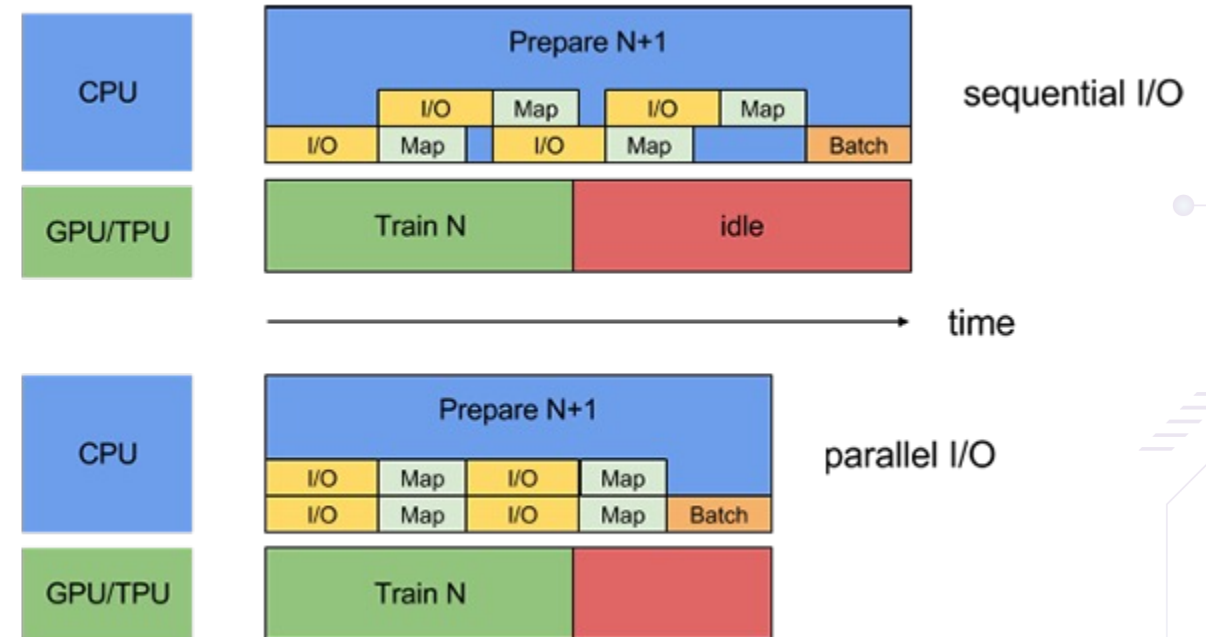
Esto es precisamente lo que permitirá realizar Pipelines.



2.6 Pipeline de TensorFlow

¿Qué es un pipeline? Es una secuencia lineal de módulos especializados diseñados para absorber, limpiar y preparar información para su consumo. O sea, todo el proceso de cargar los datos y dejarlos bonitos para que nuestra red neuronal toda delicada los pueda procesar.

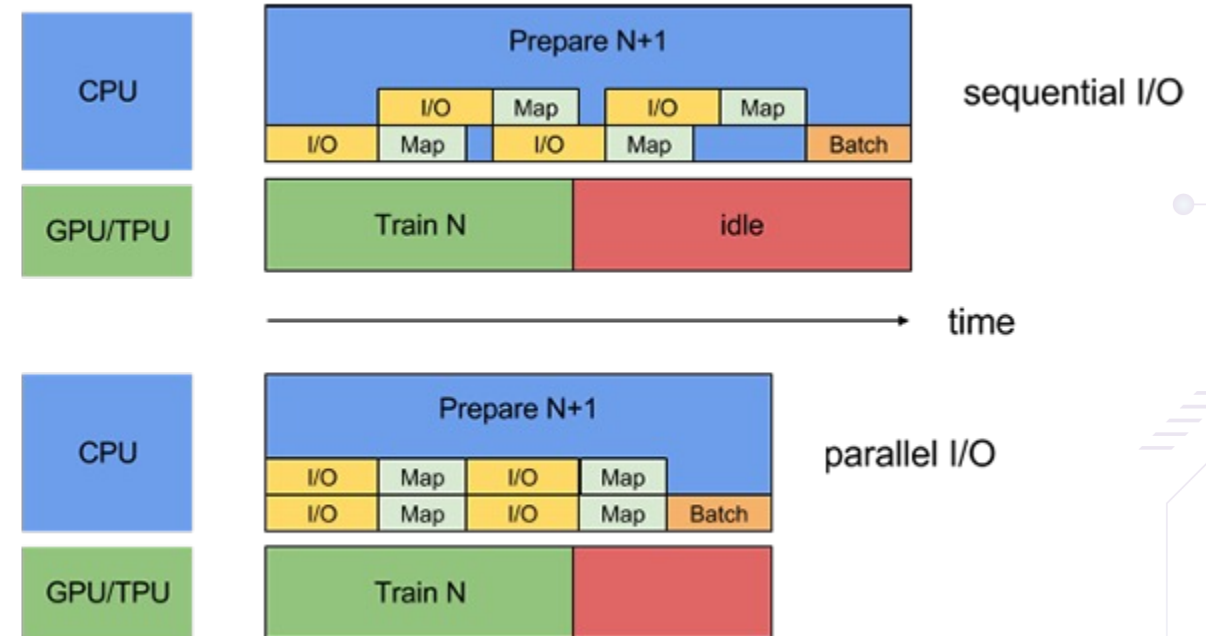
Para poder construir pipelines utilizando Tensorflow, necesitas usar el framework TF.Data. Dentro de este, la clase principal es TF.Data.Dataset.



2.7 Pipeline de TensorFlow

Lo primero antes de cargar las imágenes al modelo es asegurarse de que vienen limpias. No queremos imágenes de baja calidad, y definitivamente no queremos imágenes de cosas que parecen hot dogs pero están lejos de serlo.

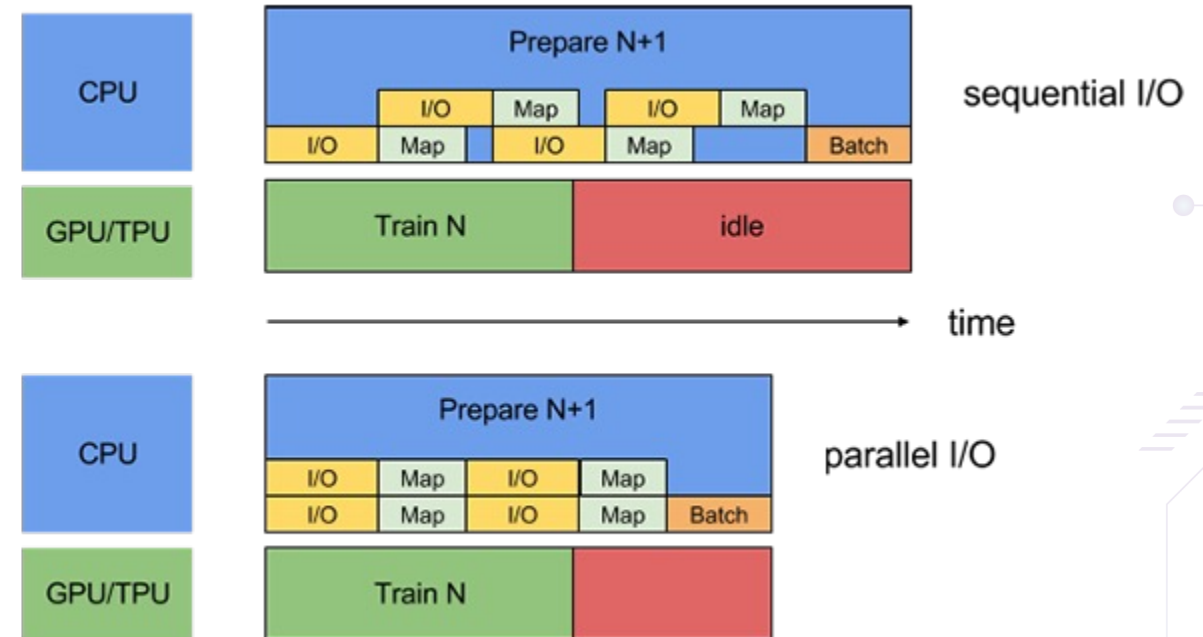
Por ello, **TF.Dataset** tiene montón de herramientas que te pueden ayudar a filtrar imágenes, por ejemplo esta imagen determinamos que parecía hot dog comestible, pero resultó ser el Tío Leonardo, así que la podemos descartar con TensorFlow.



2.8 Pipeline de TensorFlow

Lo primero antes de cargar las imágenes al modelo es asegurarse de que vienen limpias. No queremos imágenes de baja calidad, y definitivamente no queremos imágenes de cosas que parecen hot dogs pero están lejos de serlo.

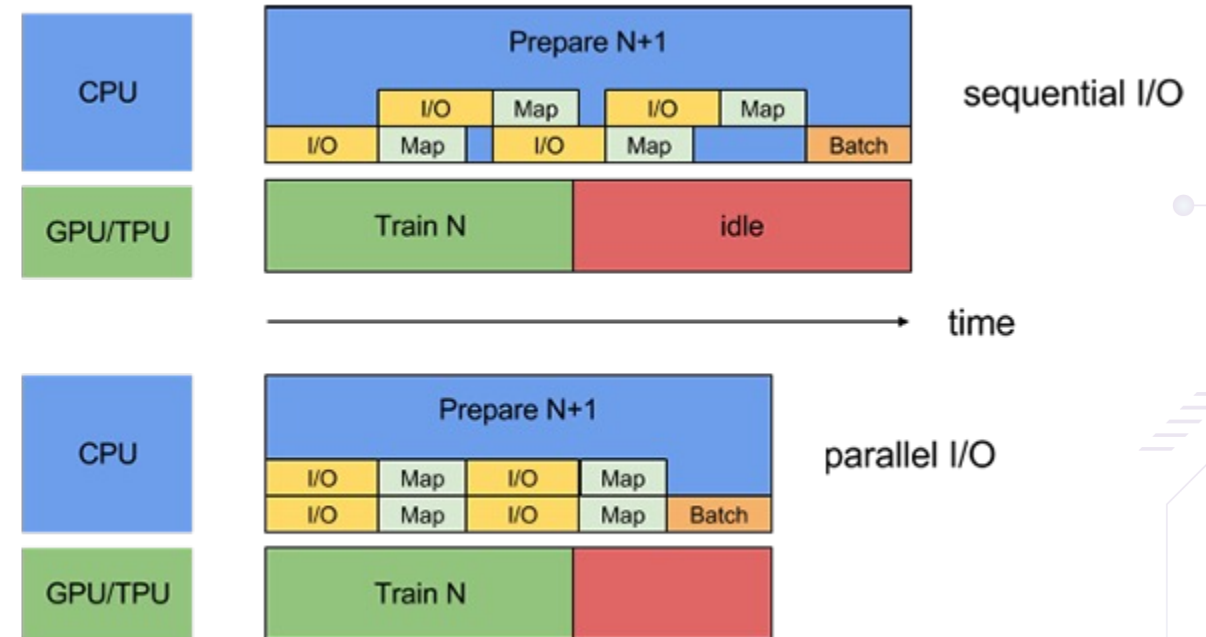
Por ello, **TF.Dataset** tiene montón de herramientas que te pueden ayudar a filtrar imágenes, por ejemplo esta imagen determinamos que parecía hot dog comestible, pero resultó ser el Tío Leonardo, así que la podemos descartar con TensorFlow.



2.9 Pipeline de TensorFlow

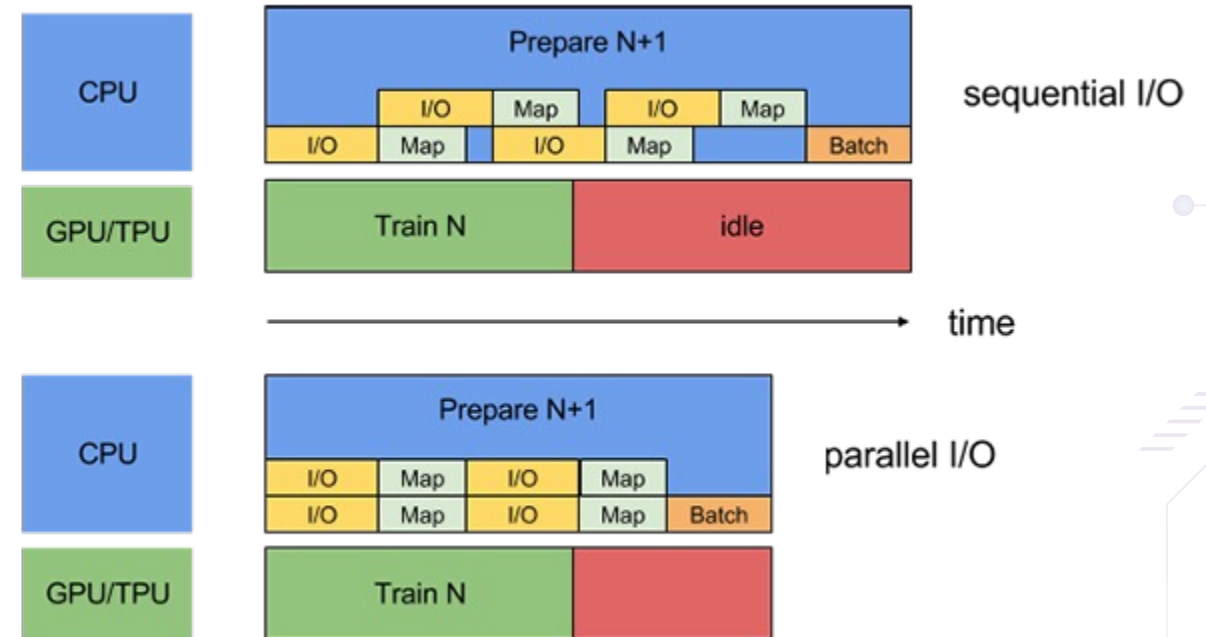
Otra tarea común de limpieza es escalar todos los valores. Por ejemplo, las imágenes están hechas de píxeles con valores RGG, los valores RGB van de 0 a 255, así que hay que dividir todos estos valores entre 255 para que funcionen las cosas.

A todo este proceso se le conoce como **ETL – Extract Transform Load**. La parte de Extract la veremos al principio, en la cual vamos a extraer datos de 20 archivos de CSV diferentes para volver a armar el set de datos de casas en California.



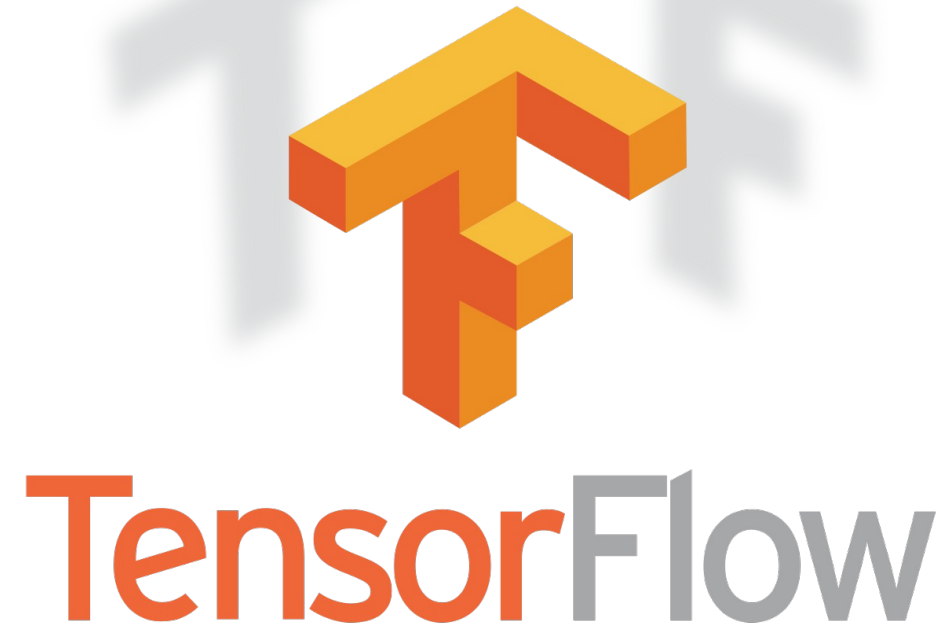
2.10 Pipeline de TensorFlow

El siguiente paso, **Transform**, es para limpiar esos datos. Vamos a estar preparando nuestros datos de California (pero la verdad ya están limpios) y vamos a ver un ejemplo donde cargamos unas cuantas imágenes usando **TF.dataset** desde nuestra computadora. Y, por último, Load, es cuando ya por fin cargamos nuestros datos a memoria, ya sea en batches o todos y nos lanzamos directamente a trabajar.



2.11 Pipeline de TensorFlow

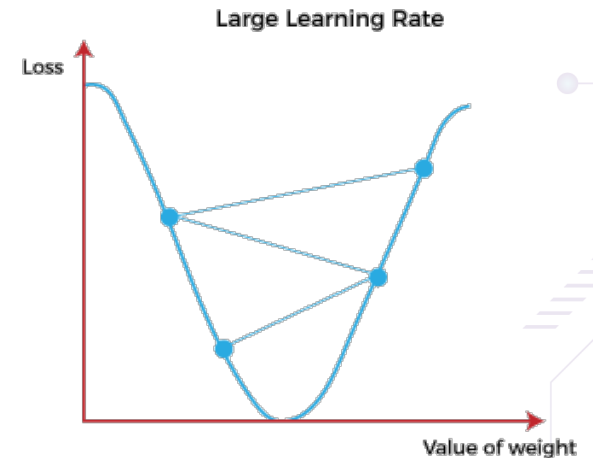
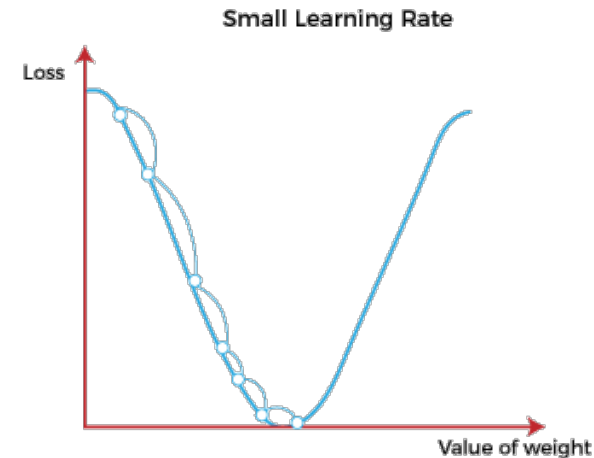
Así que, en resumen, el pipeline de Tensorflow te ofrece 2 grandes beneficios: en primera, puedes manejar cantidades enormes de datasets muy fácilmente, y segundo puedes transformar tus datos para limpiarlos y dejarlos listos para que tu red neuronal los adopte.



2.12 Barajar los Datos

Como sabemos, el Descenso del Gradiente Gradient Descent funciona mejor cuando las instancias del conjunto de entrenamiento son independientes y están distribuidas de manera idéntica. Una forma sencilla de garantizar esto es barajar las instancias. Para esto, puedes usar el método `shuffle()`.

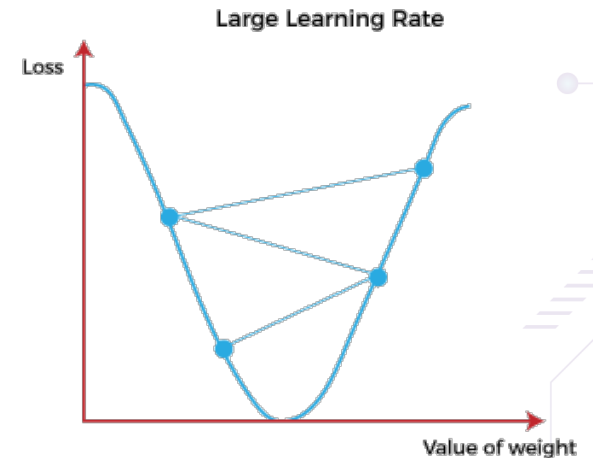
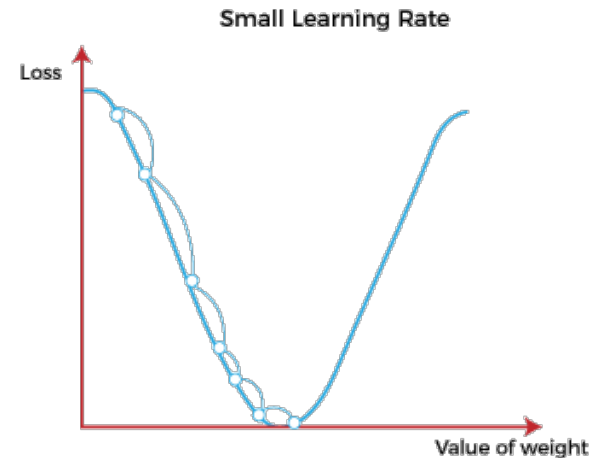
Este método creará un nuevo conjunto de datos que comenzará llenando un búfer con los primeros elementos del conjunto de datos de origen, luego, cada vez que se le pida un elemento, extraerá uno aleatoriamente del búfer y lo reemplazará con uno nuevo de el conjunto de datos de origen, hasta que haya iterado por completo a través del conjunto de datos de origen.



2.13 Barajar los Datos

En este punto, `shuffle()` continúa extrayendo elementos aleatoriamente del búfer hasta que se vacía. Para esto, se debe especificar el tamaño del búfer, y es importante que sea lo suficientemente grande o, de lo contrario, no servirá para mucho.

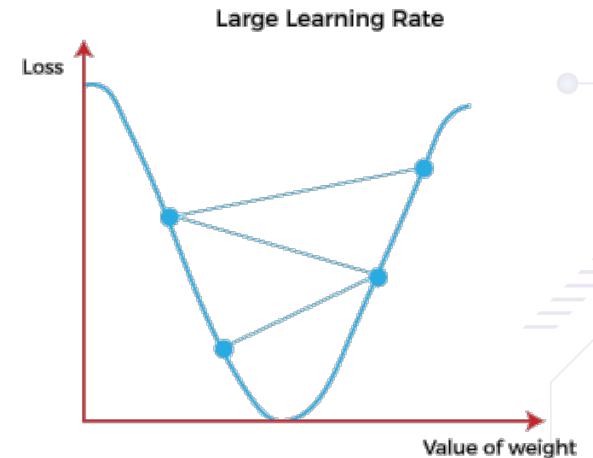
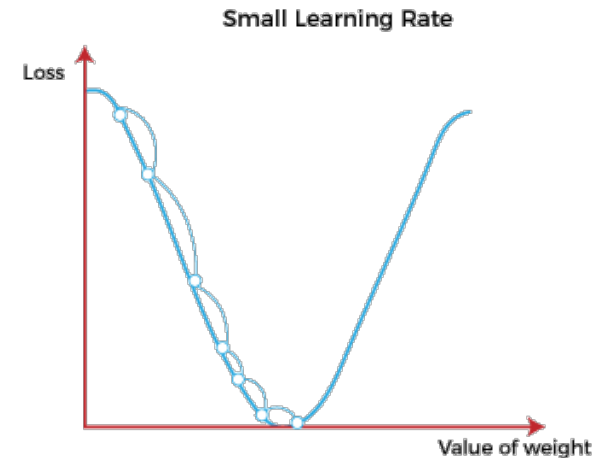
Y aunque te quieras emocionar con el tamaño de búfer, se debe de cuidar que no exceda la cantidad de RAM que tienes a tu disposición e incluso si tienes suficiente, no hay necesidad de ir más allá para despilfarrar RAM.



2.14 Barajear los Datos

Para un conjunto de datos grande que no cabe en la memoria, este enfoque simple de búfer y barajeado puede no ser suficiente, ya que el búfer será pequeño en comparación con el conjunto de datos. Una solución es dividir los datos de origen en varios archivos y luego leerlos en un orden aleatorio durante el entrenamiento.

Sin embargo, las instancias ubicadas en el mismo archivo seguirán estando cerca unas de otras. Para evitar esto, puede elegir varios archivos al azar y leerlos simultáneamente, intercalando sus líneas. Luego, además de eso, puede agregar un búfer de barajado usando el método `shuffle()`. Si todo esto parece mucho trabajo, no te preocupes, la computadora hará todo el trabajo



2.15 Prefetching

Al llamar a `prefetch()` al final, estamos creando un conjunto de datos que hará todo lo posible para estar siempre un lote por delante. En otras palabras, mientras nuestro algoritmo de entrenamiento está trabajando en un lote, el conjunto de datos ya estará trabajando en paralelo para preparar el próximo lote. Esto puede mejorar drásticamente el rendimiento.

Si también nos aseguramos de que la carga y el preprocesamiento sean de subprocesos múltiples, podemos explotar múltiples núcleos en la CPU y, con suerte, hacer que la preparación de un lote de datos sea más corta que ejecutar un paso de entrenamiento en el GPU: de esta manera, la GPU se utilizará casi al 100 %) y el entrenamiento será mucho más rápido.



2.16 Prefetching

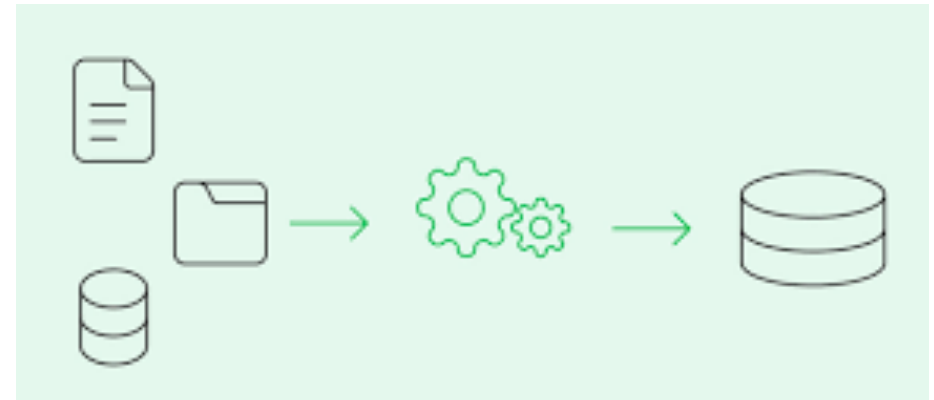
Si el dataset es lo suficientemente chico para caber en memoria, significativamente puedes acelerar el entrenamiento utilizando el caché del dataset para guardar todo en la RAM. Ahora deberías de hacer esto después de cargar y procesar los datos, pero antes de hacer el barajeo, el repetido, el batch y demás.

De esta manera cada instancia sólo va a ser procesada una vez, pero los datos van a seguir siendo barajados de manera diferente en cada época y en el siguiente lote va a ser preparado.



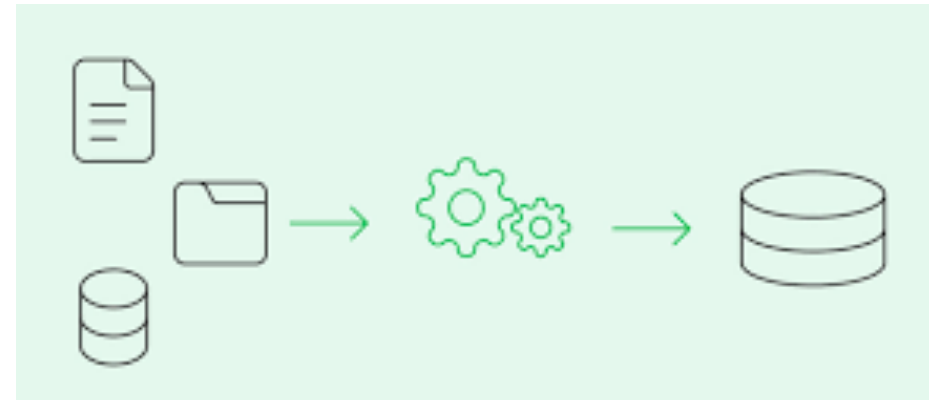
2.17 Pre-procesar Imágenes

Todo lo que hicimos hasta el momento fue de texto. Primero cargamos datos de una lista que ya venía armada, luego cargamos los datos de housing de California pero vamos a ver qué onda con imágenes porque finalmente mucho de lo que tiene que ver con con redes neuronales consiste en clasificación de imágenes.



2.18 Pre-procesar Imágenes

Vamos a revisar cómo podemos estar cargando las imágenes para hacer nuestra aplicación de si es un hot dog o no es. Para ello tienen una carpeta con un montón de imágenes de diferentes hot dogs y de diferentes comidas. No vamos a armar el clasificador, simplemente vamos a ver cómo podríamos cargar estas imágenes directamente de un dataset.



2.19 Pre-procesar Imágenes

Comencemos armando un nuevo dataset que se llame imágenes:

```
images_ds=tf.data.Dataset.list_files("HotDogs/*/*",  
shuffle = False)
```

Listo, ya tenemos ahí nuestra carga pero fíjense qué es lo que se cargó:

```
for file in images_ds.take(10):  
    print(file.numpy())
```

```
b'HotDogs/COMIDAS/00.jpg'  
b'HotDogs/COMIDAS/09.webp'  
b'HotDogs/COMIDAS/10.jpg'  
b'HotDogs/COMIDAS/11.jpg'  
b'HotDogs/COMIDAS/12.png'  
b'HotDogs/COMIDAS/13.webp'  
b'HotDogs/COMIDAS/14.webp'  
b'HotDogs/COMIDAS/17.jpg'  
b'HotDogs/COMIDAS/18.jpg'  
b'HotDogs/COMIDAS/19.jpg'
```

2.20 Pre-procesar Imágenes

Si observamos, lo que está cargando son los directorios de dónde están las imágenes, no cargamos la imagen como tal entonces nuestro data set se compone básicamente de direcciones. Tenemos que ir más allá y decirle de esa dirección jala tanto la imagen como la etiqueta porque no sólo queremos la imagen queremos pues básicamente saber si es hot dog o es otro tipo de comida.

```
b'HotDogs/COMIDAS/00.jpg'  
b'HotDogs/COMIDAS/09.webp'  
b'HotDogs/COMIDAS/10.jpg'  
b'HotDogs/COMIDAS/11.jpg'  
b'HotDogs/COMIDAS/12.png'  
b'HotDogs/COMIDAS/13.webp'  
b'HotDogs/COMIDAS/14.webp'  
b'HotDogs/COMIDAS/17.jpg'  
b'HotDogs/COMIDAS/18.jpg'  
b'HotDogs/COMIDAS/19.jpg'
```

2.21 Pre-procesar Imágenes

Lo primero que hay que hacer es conocer la cantidad de imágenes que traemos:

```
image_count = len(images_ds)
image_count
```

Son 102 imágenes que vamos a barajear con un búfer de 200 aunque este sobrado (solo son 102 imágenes).

```
images_ds = images_ds.shuffle(200)
```

Tomamos 10 imágenes:

```
for file in images_ds.take(10):
    print(file.numpy())
```

```
b'HotDogs/COMIDAS/87.jpg'
b'HotDogs/COMIDAS/29.jpeg'
b'HotDogs/COMIDAS/38.jpg'
b'HotDogs/HOTDOGS/hots.webp'
b'HotDogs/COMIDAS/53.jpg'
b'HotDogs/COMIDAS/22.webp'
b'HotDogs/HOTDOGS/p.webp'
b'HotDogs/COMIDAS/26.jpg'
b'HotDogs/COMIDAS/17.jpg'
b'HotDogs/COMIDAS/33.htm'
```

Cada vez que se ejecuta, estará dando diferentes arreglos

2.22 Pre-procesar Imágenes

Ahora, vamos a armar un set de entrenamiento y un set de prueba. El set de entrenamiento está sencillo:

```
train_size = int(image_count*.8)
train_ds = images_ds.take(train_size)
test_ds = images_ds.skip(train_size)
```

Entonces ya armamos nuestro set de entrenamiento y nuestro set de prueba. Si observamos, lo que le estoy diciendo es que armará un set de entrenamiento que fuera del 80% del total de las imágenes y le pusimos int para que redondeará el resultado.

2.23 Pre-procesar Imágenes

Por otro lado, vamos a checar los largos de los sets:

```
len(test_ds)
```

Vamos haciendo la parte importante, hay que traer las etiquetas y hay que traer la imagen como tal. La etiqueta está bien sencilla, vamos a armar una función para eso:

```
def get_label(file_path):  
    parts = tf.strings.split(file_path, os.path.sep)  
    return parts[-2]
```

Entonces lo que estoy pidiendo en esta función es que me separe el filepath (el nombre de la dirección) porque la etiqueta finalmente es el folder en donde están viviendo las imágenes. Ejecutamos y vamos probando la función de la imagen:

```
for file in images_ds.take(10):  
    print(get_label(file))
```

Ya me está dando mi etiqueta correctamente por imagen:

```
tf.Tensor(b'HOTDOGS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)  
tf.Tensor(b'HOTDOGS', shape=(), dtype=string)  
tf.Tensor(b'HOTDOGS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)  
tf.Tensor(b'HOTDOGS', shape=(), dtype=string)  
tf.Tensor(b'COMIDAS', shape=(), dtype=string)
```

2.24 Pre-procesar Imágenes

Ahora, necesitamos una función adicional para procesar la imagen. La función es bastante sencilla. Si recordamos, TensorFlow no lee las imágenes como nosotros los seres humanos, el programa va a leer cada píxel, se fijará en la locación del píxel, la profundidad de los colores RGB, etc.

```
def process_image(file_path):
    label = get_label(file_path)
    img = tf.io.read_file(file_path)
    img = tf.image.decode_jpg(img)
    img = tf.image.resize(img, [128,128])
    return img, label

img,label =
process_image("HotDogs\\COMIDAS\\00.jpg")
```

Si le pido que muestre la imagen:

`img`

```
<tf.Tensor: shape=(128, 128, 3), dtype=float32, numpy=
array([[ [2.52832031e+02, 2.51832031e+02, 2.47832031e+02],
        [2.51437500e+02, 2.50437500e+02, 2.46437500e+02],
        [2.52000000e+02, 2.52000000e+02, 2.50000000e+02],
        ...,
        [1.91591064e+02, 1.48591064e+02, 1.16591064e+02],
        [1.81394531e+02, 1.43394531e+02, 1.07394531e+02],
        [1.84683594e+02, 1.45683594e+02, 1.14683594e+02]],

        [ [2.54000000e+02, 2.54000000e+02, 2.52000000e+02],
        [2.52000000e+02, 2.52000000e+02, 2.50000000e+02],
        [2.53000000e+02, 2.52000000e+02, 2.47000000e+02],
        ...,
        [1.95441406e+02, 1.59937500e+02, 1.27937500e+02],
        [1.90496094e+02, 1.54496094e+02, 1.22496094e+02],
        [1.90086670e+02, 1.53086670e+02, 1.25078857e+02]],

        [ [2.52809814e+02, 2.52839844e+02, 2.50749756e+02],
        [2.53839844e+02, 2.53749756e+02, 2.55000000e+02],
        [2.54212646e+02, 2.49902344e+02, 2.39187500e+02],
        ...,
        [1.85732178e+02, 1.51732178e+02, 1.23732178e+02],
        [1.88519531e+02, 1.53519531e+02, 1.23519531e+02],
        [1.84157471e+02, 1.49157471e+02, 1.19157471e+02]],

        ...,

        [ [2.36130127e+02, 2.28522217e+02, 2.17899658e+02],
        [2.36827637e+02, 2.20722656e+02, 1.89874756e+02],
        [2.39062500e+02, 2.25777344e+02, 2.10080322e+02],
        ...,
        [7.93750000e+00, 3.93750000e+00, 9.37500000e-01],
        [9.00000000e+00, 4.00000000e+00, 0.00000000e+00],
```

2.25 Pre-procesar Imágenes

Recuerden que la imagen es una matriz de 128 por 128. Vamos a “mapear” nuestros datos con las nuevas funciones básicamente transformarlos porque recuerden que map lo que hace es transformar:

```
train_ds = train_ds.map(process_image)
test_ds = test_ds.map(process_image)
```

Ahora sí, continuemos con la revisión de otra imagen del set de entrenamiento, nada más para saber como quedo.

```
for image,label in train_ds.take(1):
    print("Image", image)
    print("Label", label)
```

```
Image tf.Tensor(
[[[ 11.  14.  65.]
 [ 15.  20.  75.]
 [ 12.  17.  57.]
 ...
 [150. 251. 237.]
 [181. 253. 249.]
 [224. 254. 254.]]

[[ 16.  19.  72.]
 [ 16.  18.  66.]
 [ 49.  57.  94.]
 ...
 [194. 254. 246.]
 [241. 250. 255.]
 [237. 253. 252.]]

[[ 13.  17.  65.]
 [ 15.  14.  54.]
 [ 14.  22.  68.]
 ...
 [232. 254. 252.]
 [206. 255. 249.]
 [165. 251. 242.]]

...

[[ 87. 171. 156.]
 [ 87. 171. 156.]
 [ 84. 168. 153.]
 ...
 [165. 251. 242.]
```

2.26 Pre-procesar Imágenes

Cada uno de estos valores, es un valor RGB de algún píxel en cuestión y esto es lo que va a leer la red neuronal. Si nosotros usáramos matplotlib podríamos ver la imagen como tal.

Por último, si a nosotros nos interesara podríamos escalar la imagen para que sea entrenable según el algoritmo que estemos utilizando:

```
def scale(image,label):  
    return image/255, label
```

Esto es para que los valores RGB estén entre 0 y uno y nos permite hacer el entrenamiento de una mejor manera.

```
train_ds = train_ds.map(scale)  
for image, label in train_ds.take(1):  
    print("****Image: ",image.numpy()[0][0])  
    print("****Label: ",label.numpy())
```

Aplicamos esto al set de entrenamiento:

```
train_ds = train_ds.map(scale)
```

Luego, vamos a checar algún dato con el escalamiento aplicado.

```
for image, label in train_ds.take(1):  
    print("****Image: ",image.numpy()[0][0])  
    print("****Label: ",label.numpy())
```

```
****Image: [0.74969363 0.7143995 0.68694854]  
****Label: b'COMIDAS'
```

Si se fijan, ahí están los valores entre 0 y 1.

3.1 Introducción

Como bien ya lo hemos estado viendo en los capítulos anteriores, las redes neuronales se han posicionado como una de las herramientas de mayor importancia en la industria, debido a sus distintas aplicaciones.

El interés de tener cada vez capas ocultas más densas en las redes para el reconocimiento de patrones ha llevado a los científicos de datos a desarrollar una de las redes neuronales profundas más populares, las redes neuronales convolucionales (CNN – por sus siglas en inglés). Este nombre proviene de la operación matemática en la que se basa el modelo, que es la convolución. Esta red está compuesta de varias capas como la de convolución y la de pooling entre otras de las que estaremos hablando en un futuro.



$1820 \times 1080 \times 3 \gg$ muchos pixeles

3.2 Introducción

Las CNN optimizan perfecto en problemas de machine learning (ML), especialmente en los relacionados con el procesamiento de imágenes, como la clasificación de imágenes, visión por computadora y el procesamiento de lenguaje natural (NLP)

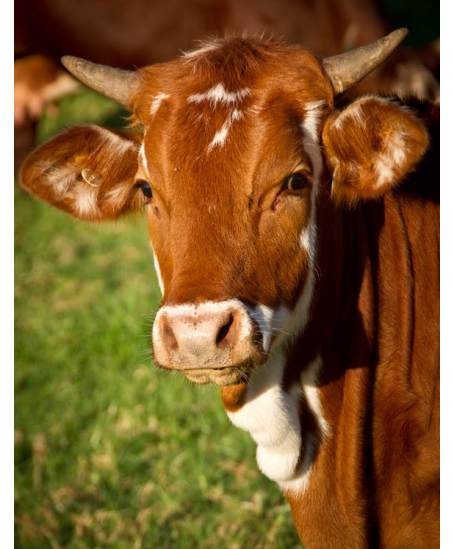
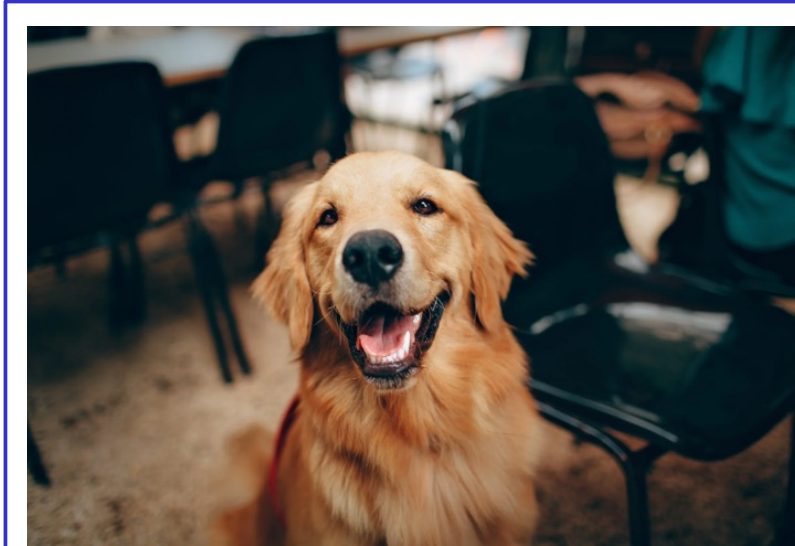
A continuación, explicaremos de la manera más sencilla de que trata esta arquitectura, sí que concéntrate y pon atención.



3.3 Introducción

¿Qué animal ves en esta foto?
¿Y en esta foto?
¿Y en esta otra?

Son matrices
de valores
¿Como diferenciarlos?

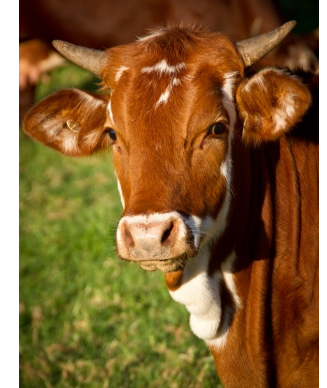


© DK 2003

3.4 Introducción

Reconocer estos animales es una tarea trivial que cualquier niño que apenas sabe hablar ya puede completar con éxito.

Pero para una computadora esto es un martirio – es solo hasta la última década (2010s) que las computadoras pueden identificar objetos en imágenes con algo de confiabilidad. Lo mismo aplica para audios – solo hasta hace poco las computadoras pueden entender lo que les estas diciendo sin trabarse (mucho).



© DK 2003

3.5 Introducción

¿Por qué para los humanos se nos da tan fácil reconocer imágenes y escuchar palabras?

La realidad es que nuestros ojos son máquinas complejas de preprocesamiento de datos que se encargan de adornar las imágenes que recibimos con toda clase de características para que cuando nos llegue al cerebro, podamos decir “Obviamente es Cell en el capítulo 167 de Dragon Ball Z lanzado en 1992”.



3.6 Introducción

Lo mismo pasa con nuestros oídos al momento de escuchar sonidos.

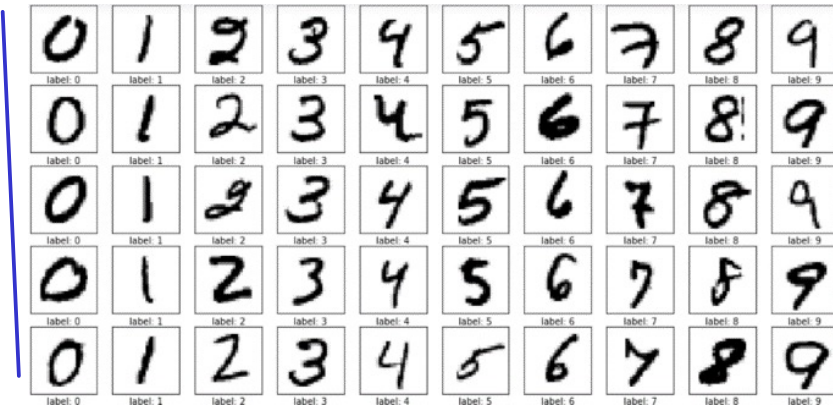
El audio entra en nuestro canal auditivo, es preprocesado por un sinnúmero de nervios y otros tejidos especializados que se encargan de pasar la señal a través de un aparato producto de 4 billones de años de evolución, maravilla de la creación, para que tu escuches lo más majestuoso que la música humana tiene que ofrecerte:

“Ella Perrea Sola”

3.7 Introducción

El punto al que queremos llegar con todo esto es que: la manera en la que funcionan nuestros módulos sensoriales humanos es muy similar a la manera en la que podemos lograr que una computadora “vea”, “escuche”, o “lea”.

Las Redes Neuronales Convolucionales (CNN por sus siglas en inglés) surgen en los 80s gracias al estudio de la corteza visual del cerebro. Al principio solo se usaban para el reconocimiento de imágenes muy sencillas – literal nuestros dígitos escritos a mano que tanto amamos de MNIST.



3.8 Arquitectura de la corteza visual

Que exista un sistema mediante el cual los seres vivos en el mundo puedan ver es casi casi un milagro. El complejo visual de insectos, aves, mamíferos y seres humanos en particular es increíble, y el hecho que el relojero ciego de la evolución lo haya logrado diseñar a pura prueba y error es todavía más inaudito.

Desde trilobitos antiguos que tenían simples sensores que les decían si estaban en un lugar con mucha luz o poca luz, hasta la visión humana que nos permite ver toda clase de videos en tiktok.

airplane



automobile



bird



cat



deer



dog



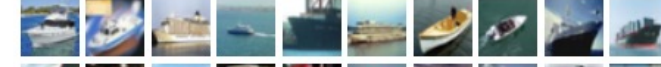
frog



horse



ship



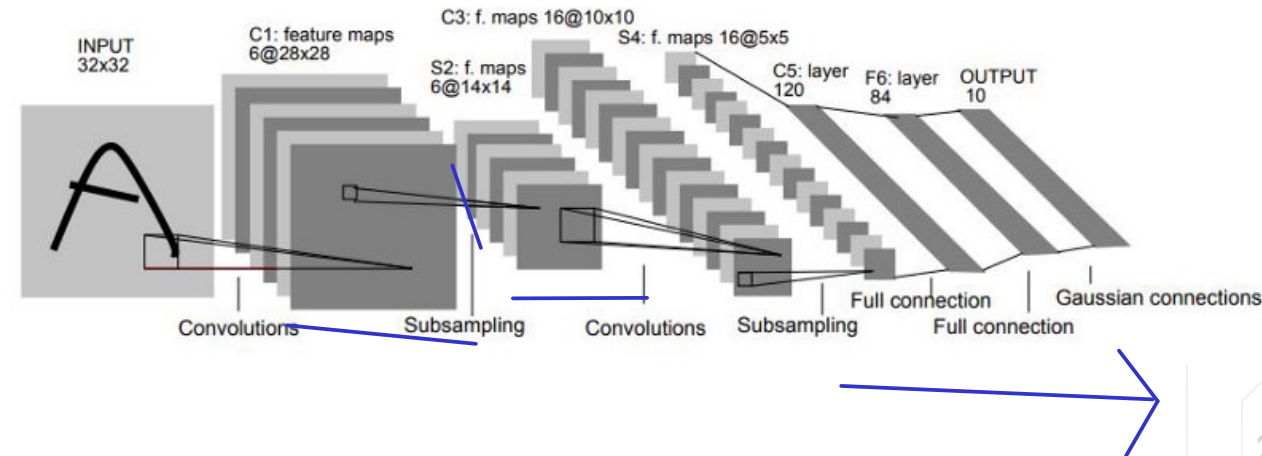
truck



3.9 Arquitectura de la corteza visual

En 1998 se introduce la primera red neuronal convolucional que tenía aplicación práctica – LeNet-5 inventada por Yann LeCun (lo conocerán por su función de inicialización que vimos en el capítulo 2). Se usó bastante por los bancos para reconocer números en cheques escritos a mano – básicamente la tarea que esperas de una red que estudia MNIST.

LeNet-5 es una red neuronal con muchos elementos que ya conoces – capas completamente conectadas, funciones de activación sigmoide, etc. Pero también tiene 2 elementos fundamentales en las redes convolucionales que no conoces – las capas convolucionales y las capas de pooling.

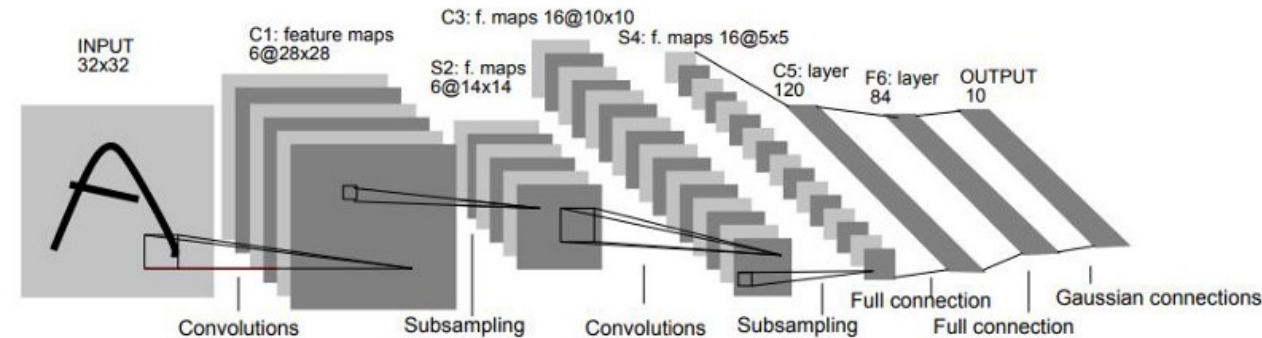


3.10 Arquitectura de la corteza visual

¿Por qué usaremos estas capas en vez de usar una red neuronal profunda con capas densas totalmente conectadas? Bueno, ya vimos, de entrada, que eso no sirve. Nuestra red que quería clasificar las imágenes todas chafas de CIFAR10 nos estaba dando un 50% aproximado de éxito.

Simplemente tiene el problema que la red se vuelve muy grande muy rápido y eso impide que pueda progresar – por ejemplo, una imagen de 100 x 100 píxeles tiene 10,000 píxeles. Y si la pasamos a una capa densa de 1000 neuronas (poco en realidad, porque son 10 veces menos neuronas que píxeles) entonces tenemos un total de ¡10 millones de conexiones! ¡En una sola capa!

Los CNNs resuelven este problema usando capas parcialmente conectadas. Veamos cómo.



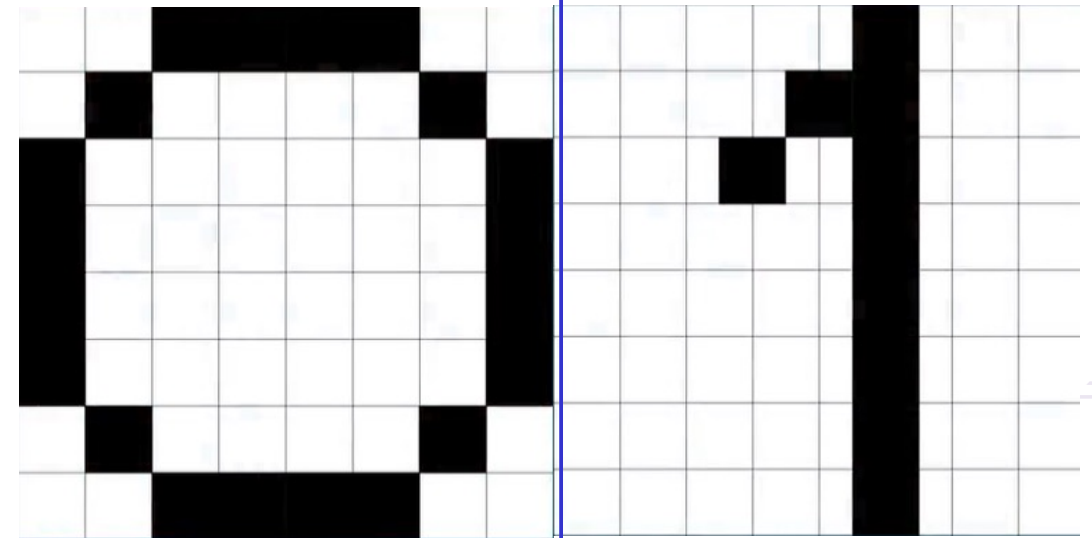
3.11 Funcionamiento de una Red Neuronal Convolutiva

¿Como funciona una Red Neuronal Convolutiva?

Quiero que recuerden que los conceptos clave son la Capa Convolutiva y la Capa de Pooling. Vamos a imaginarnos primero que estamos intentando clasificar imágenes realmente sencillas.

El plan es alimentar nuestra red neuronal con imágenes de “0s” y “1s” de este estilo y la computadora nos tiene que poder decir de que dígito se trata.

Dado que el 0 y el 1 están dibujados en computadora, nosotros sabemos que son básicamente un montón de píxeles en diferentes escalas de grises. En este caso, para simplificar la vida solo hay 2 colores presentes, blanco y negro.



3.12 Funcionamiento de una Red Neuronal Convolutiva

Los pixeles negros vamos a representarlos con un valor 1 y los pixeles blancos con un valor 0. No se me vayan a confundir con las imágenes! Es solo coincidencia que hayamos agarrado los números 0 y 1.

Ok, vamos comenzando, viendo cómo se puede clasificar el 0 – esta imagen está ridículamente pequeña, solo 8 x 8 pixeles. Eso significa que claro que la podríamos clasificar usando una red neuronal sencilla construida a partir de puras capas densas.

Simplemente agarramos nuestra imagen de 8 x 8 y lo convertimos en una serie de 64 neuronas (nuestra capa de Flatten), conectamos estas neuronas a una capa oculta de una sola neurona. Eso significa 64 conexiones en total.

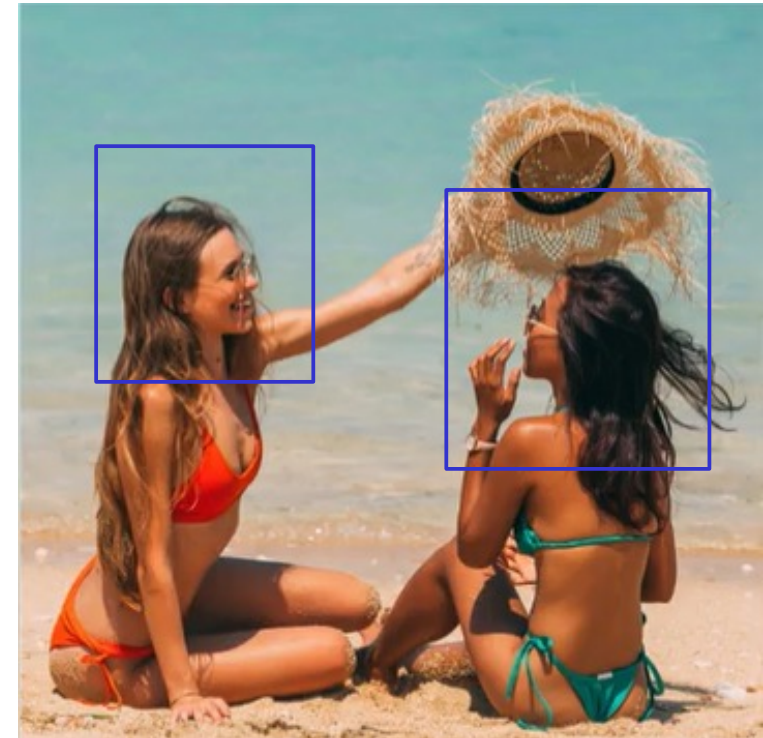
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

3.13 Funcionamiento de una Red Neuronal Convolutiva

Si fuéramos a ejecutar nuestro algoritmo, recordamos que mediante backpropagation tendríamos que calcular 64 pesos para las 64 conexiones – y ustedes ya sabrán que esto es una red ridículamente sencilla. Claro que la capa oculta debería tener más neuronas.

Pero cada neurona que agreguemos a la capa oculta son 64 conexiones más, y otras 64, y otras 64... si tuviéramos 20 neuronas en la capa oculta ya tendríamos 1280 conexiones en total.

Ahora, como la imagen es chica, sabemos perfectamente que puede funcionar ... pero ¿qué pasa si le metes esta imagen de Instagram y quieres reconocer las caras de los vacacionistas?



3.14 Funcionamiento de una Red Neuronal Convolutacional

1080x1080x3

Instagram usa imágenes de 1080 x 1080 píxeles por default – eso significa que tendrías que meter 1,166,400 valores en una capa Flatten de ese tamaño, y por cada neurona que agregues a tu capa oculta, serían ... 1 166 400 conexiones adicionales. Tu computadora te va a mentar la madre cuando la pongas a entrenar una red neuronal profunda de ese tamaño.

Ok, entonces problema número uno con detectar imágenes con una red neuronal completamente conectada (ósea con capa Densa como la que hemos trabajado hasta ahorita) es que: NO escalan bien. Mientras más grande sea la imagen, se vuelve exponencialmente más difícil el entrenamiento – y por ende nos dará peores resultados.



3.15 Funcionamiento de una Red Neuronal Convolutiva

Otro problema muy severo con las redes neuronales tradicionales es que no trabajan bien si les mueves la imagen siquiera un poco.

Volviendo a nuestro ejemplo del 0 – si lo empujamos un píxel para la izquierda la red neuronal va a decir “Hace ratito esto era un 0, pero esta imagen nueva pues no tengo idea de que se supone que estoy viendo”.

Lo cual es completamente inútil si estás intentando reconocer caras en Instagram. Esta serie de fotos evidentemente son de las mismas personas en la misma playa tomadas en la misma sesión – una red neuronal tradicional no podría identificarlos porque sus caras se movieron ligeramente. Entonces ese fue el problema número 2.



3.16 Funcionamiento de una Red Neuronal Convolutiva

Por último – un tercer problema que nos encontramos es el de los pixeles correlacionados. Por ejemplo, en esta imagen de Instagram tenemos playa casi blanca del Caribe y horizonte azul. En general los pixeles café-blancos de la arena van a estar junto a otros pixeles café-blancos. Los pixeles azules del horizonte van a estar junto a otros pixeles azules en el cielo.

Entonces sería útil aprovechar esta información para que la red neuronal capte que la imagen está básicamente dividida en 2 áreas muy bien demarcadas. Una red neuronal tradicional no va a aprovechar esta facilidad para entrenarse más fácilmente.



3.17 Funcionamiento de una Red Neuronal Convolutiva

Las CNNs tienen varias ventajas:

1. Reducen el número de nodos de ingreso – no necesitamos 64 en el caso de estas imágenes – o ~~1,166,000~~ en el caso de las fotos de Instagram.
2. Toleran pequeños cambios en la posición de las imágenes. Eso significa que este 0 me lo ubica incluso si esta desplazada a la izquierda o la derecha. O las fotos de Instagram pueden identificar a la misma persona en diferentes poses.
3. Toman ventaja de las correlaciones que vemos en imágenes complejas, así que son más eficientes para la clasificación de estas.

3.18 Funcionamiento de una Red Neuronal Convolutiva

Bien, vamos a intentar pasar nuestro “0” por una CNN.

El primer paso que hace nuestra red convolutiva es aplicar una cosa que se llama FILTRO. Pongan atención y tomen nota, este concepto es importante.

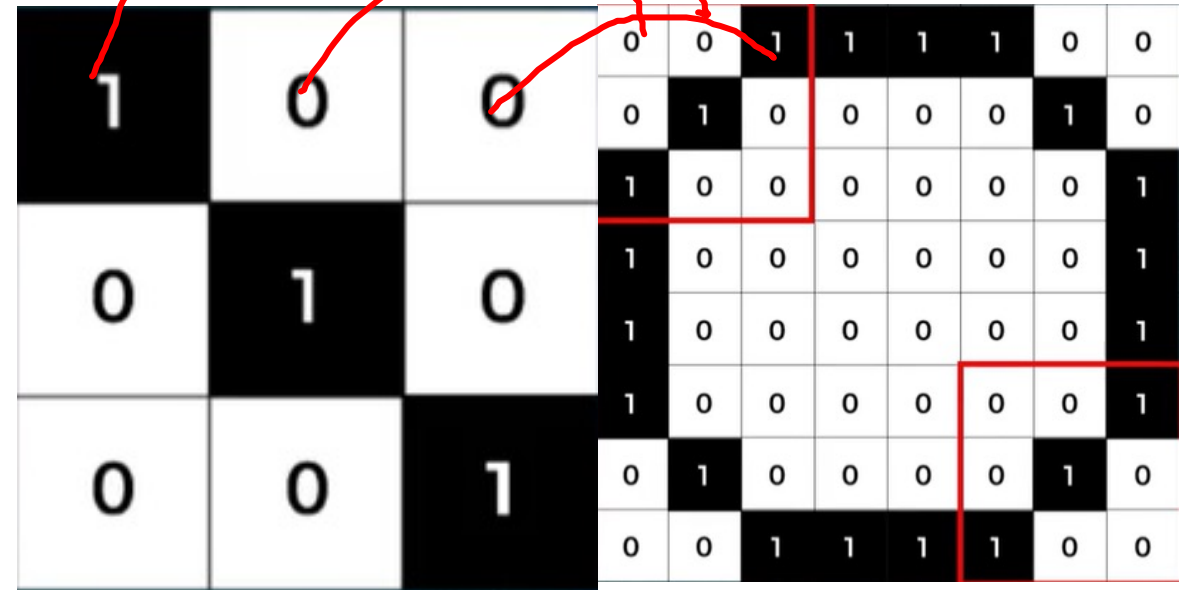
Es más fácil explicar que es un filtro mediante un ejemplo. Un filtro es simplemente un cuadro más pequeño, como de 3x3 (generalmente) que le dice a la red neuronal que tiene que estar buscando – figuras, patrones, colores, orillas.

0	0	1
0	1	0
1	0	0

3.19 Funcionamiento de una Red Neuronal Convolutiva

En este caso nuestro filtro es simplemente esta diagonal, que nos ayudaría a ubicar estas 2 partes del 0.

Aunque no lo veremos en el ejemplo, también podríamos tener un filtro como este, que nos ayudaría a ubicar estas 2 partes del 0.



1	0	0	0	0	1	1	1	1	0	0
0	1	0	0	1	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	1	0
0	0	0	0	1	0	0	0	0	0	0

3.20 Funcionamiento de una Red Neuronal Convolutiva

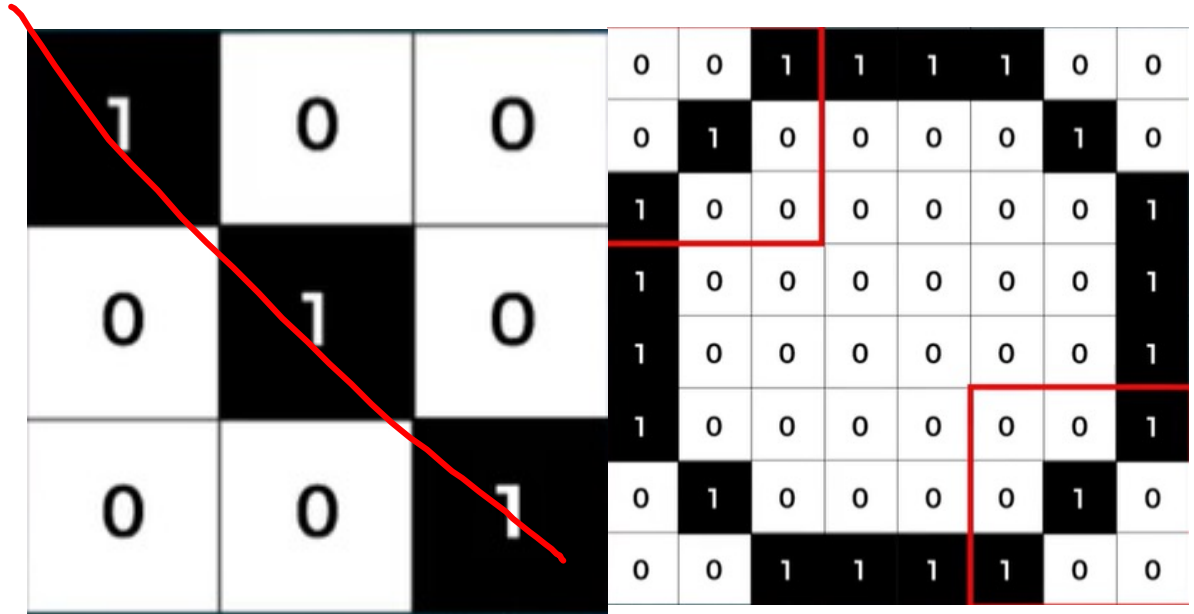
O en el caso de las fotos de Instagram para identificar caras, podríamos tener un filtro un poco más grande (digamos 20x20 píxeles) que está intentando ubicar las siluetas de ojos, o de narices, o de bocas.

Entonces el filtro se pasea por toda la imagen y le va asignando puntajes a cada área que evalúa – si catcha una imagen que se parece al filtro, entonces le va a asignar un alto puntaje a esta área.

Pero nos estamos adelantando a nosotros mismos.

¿De dónde va a inventar la computadora filtros para ojos, narices, bocas? ¿O en el caso de los 0s y 1s el filtro para la diagonal que sube hacia la derecha?

¿Se la tenemos que especificar nosotros?



3.21 Funcionamiento de una Red Neuronal Convolutiva

¡No! Esa es la parte genial de las redes neuronales. Igual que muchas otras cosas en redes, los mejores filtros posibles los va a determinar la misma computadora usando backpropagation. En el caso de nuestro 0 al principio va a armarse un filtro aleatorio, como esta colección de blancos y negros sin sentido – después de varias pasadas a través de la red neuronal, el filtro va a ir evolucionando a esta diagonal que nos va a ayudar a determinar si esto es un 0 o un uno.

1	0	1
0	0	1
1	1	0



0	0	1
0	1	0
1	0	0

3.22 Funcionamiento de una Red Neuronal Convolutiva

Lo mismo para la imagen de Instagram – va a empezar con filtros todos aleatorios, y esos filtros van a evolucionar a figuras que asemejen ojos, o narices, o bocas, o caras completas.

Ok, ¿ya tenemos nuestro filtro de diagonal, como se aplica a nuestra imagen del 0?

Fácil, sobreponemos el filtro a la imagen de ingreso en la primera esquina.

El filtro tiene ciertos valores de 0 y 1, la imagen también tiene sus valores de 0 y 1, así que cada valor sobrepuesto es multiplicado y luego sumamos todos los productos para obtener un valor final, que en este caso es 3.

1	0	1
0	0	1
1	1	0



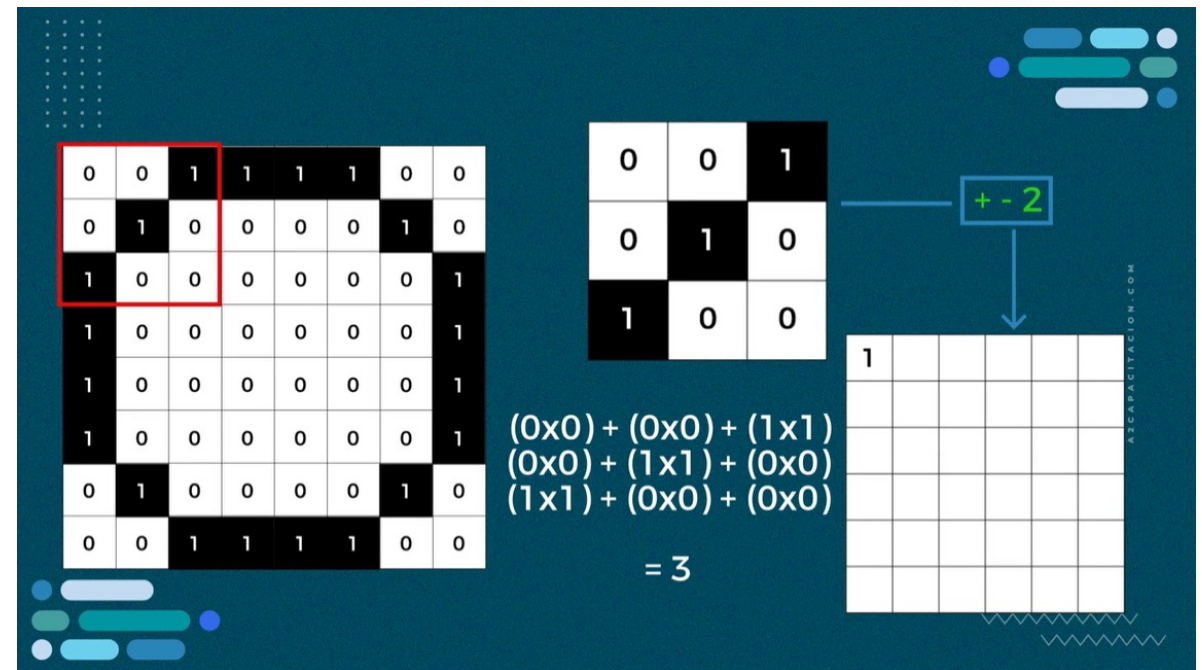
0	0	1
0	1	0
1	0	0

3.23 Funcionamiento de una Red Neuronal Convolutiva

Si recuerdan sus clases de algebra lineal, esto no es más que el producto punto del filtro, y la sección de la imagen que estamos usando ahorita.

Es en este momento que estamos aplicando la operación matemática de convolución, convolviendo el filtro con la imagen de ahí viene el nombre de redes neuronales convolucionales.

Ok, del resultado final, le sumamos nuestro termino de bias que nunca puede faltar y guardamos nuestro valor en una cosa que se llama FEATURE MAP en inglés. En español es mapa de características, pero nadie le dice así



3.24 Funcionamiento de una Red Neuronal Convolutiva

Ahora en este ejemplo vamos a deslizar nuestro filtro 1 píxel para la derecha. Esto se llama el STRIDE en inglés o zancada en español.

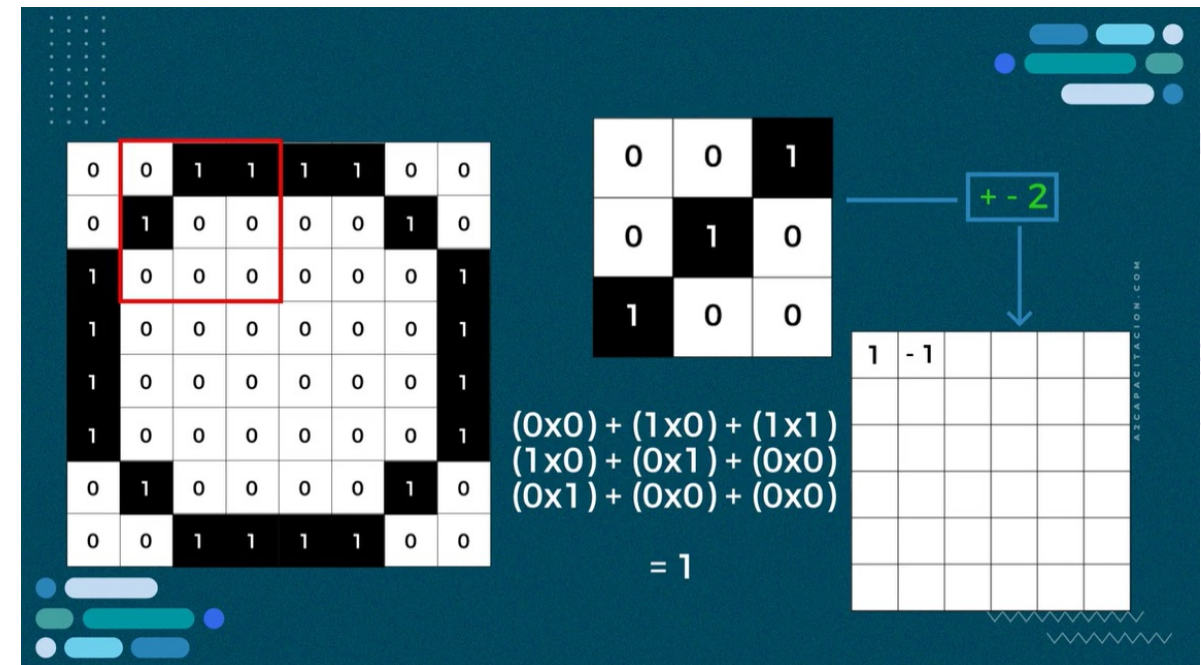
El Stride es de 1 en este caso, porque se movió un píxel, pero puedes hacer un stride de 2, 3,4,5 o lo que gustes y mandes. Por lo pronto nos quedaremos con 1 de stride.

0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

3.25 Funcionamiento de una Red Neuronal Convolutiva

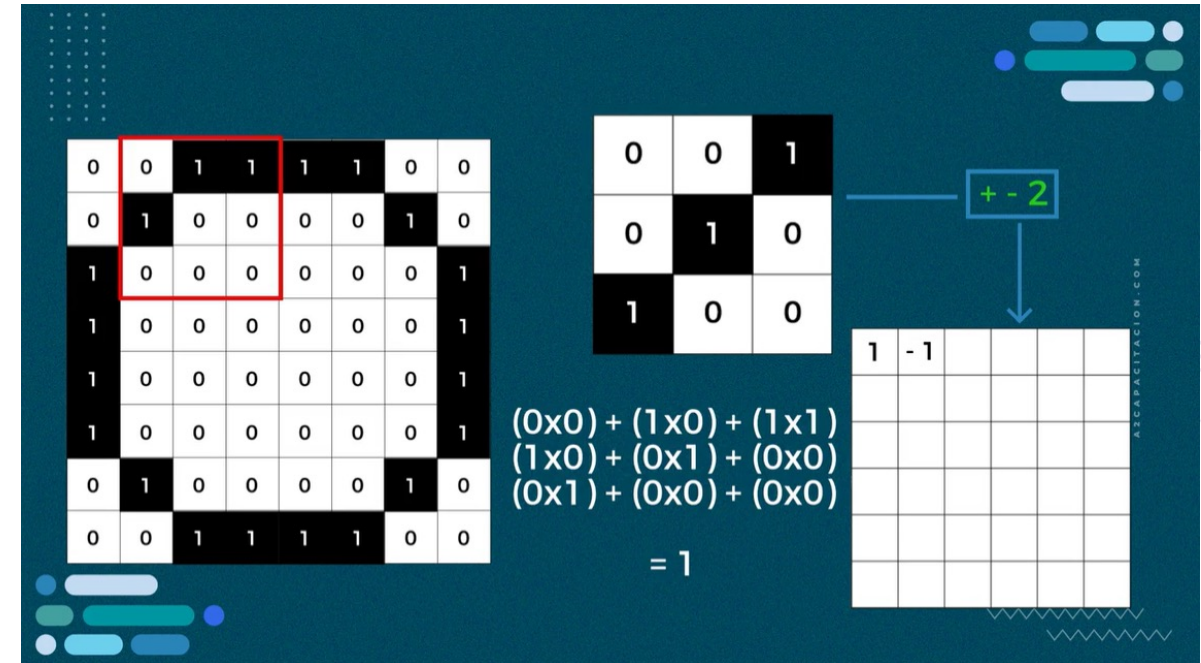
Ahora que el filtro está en su nueva posición, de nuevo calculamos el producto punto de estos pixeles, tomamos el total, le sumamos el termino de bias, y mandamos el resultado a nuestro Feature Map.

Y luego movemos el filtro de nuevo 1 píxel a la derecha, hacemos nuestro proceso que ya conocen, producto punto, sumar bias, mandar al feature map. Y así nos la llevamos hasta que pasamos el filtro por toda la imagen y hemos llenado nuestro Feature map.



3.26 Funcionamiento de una Red Neuronal Convolutiva

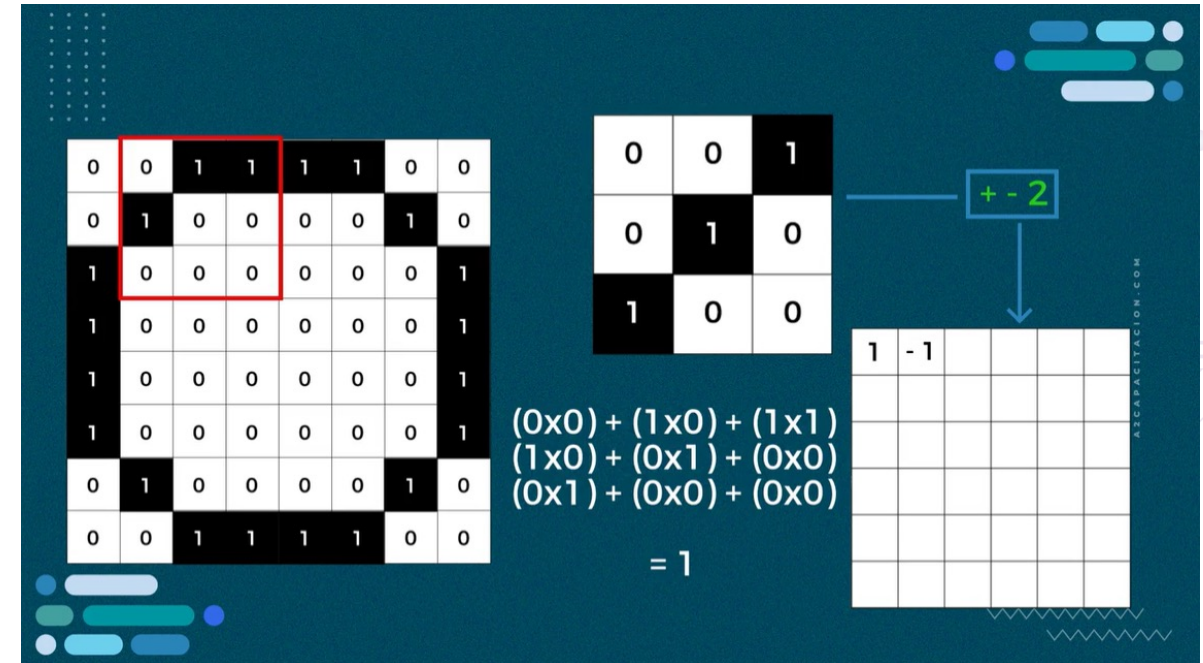
Recordemos que hicimos hasta ahorita – comenzamos con una imagen del 0, le aplicamos un filtro (que la red neuronal determinó cual era el mejor por cierto) y agregamos un término de bias a los valores que nos dieron como resultado del filtro. Estos valores los estamos guardando en un Feature Map. Recordemos que cada celda en el Feature Map corresponde a un grupo de pixeles vecinos.



3.27 Funcionamiento de una Red Neuronal Convolutiva

Esto que acabamos de hacer es cómo funciona una Capa de Convolución – y su resultado siempre es un Feature Map.

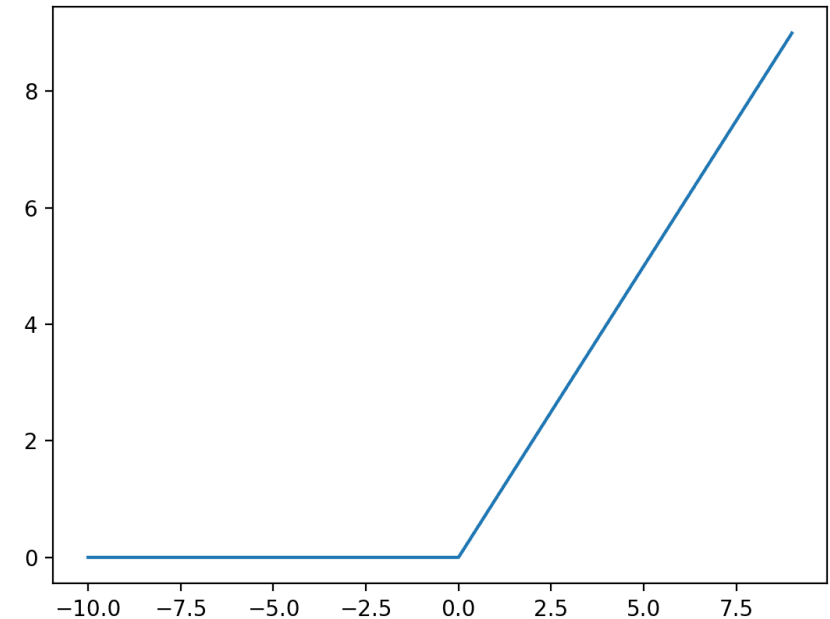
Al rato que apliquemos ConvLayers2D en TensorFlow, ya saben exactamente qué está haciendo la computadora.



3.28 Funcionamiento de una Red Neuronal Convolutiva

Ahora, al siguiente paso – vamos a agarrar los resultados de nuestro Feature Map y aplicarles una función de activación.

Recordemos que RELU es esta función – si los valores son negativos entonces se convierten en 0s y si los valores son positivos entonces se pasan tal cual como están.



3.29 Funcionamiento de una Red Neuronal Convolutiva

En este caso, todo se va a convertir a 0 excepto estos 2 puntos que son básicamente las regiones donde el filtro encontró una diagonal bien similar en la imagen del 0 original.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1

3.30 Funcionamiento de una Red Neuronal Convolutiva

Esta capa funciona con filtros también pero es diferente por lo siguiente:

Vamos a aplicar un filtro, pero ahora la meta es implementar seleccionar el valor máximo de los valores que caen dentro del filtro. Y ya, es todo. Seleccionamos el máximo no porque la red neuronal nos lo dijo, sino porque así se llama la capa, MAX pooling.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1



1	0
0	1

3.31 Funcionamiento de una Red Neuronal Convolutiva

Para ver que efectos está teniendo el max pooling – vamos de regreso a nuestra imagen de entrada y vemos que en la orilla superior izquierda coincide perfectamente con el filtro original, así que esto resulta en el valor máximo posible en el feature map.

Y luego el Max Pooling reduce esta región a el punto donde el filtro hizo su mejor chamba ajustando la imagen de ingreso. Así que Podríamos concluir que Max Pooling selecciona los puntos donde el filtro hizo el mejor trabajo de coincidir con la imagen de entrada.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1



1	0
0	1

3.32 Funcionamiento de una Red Neuronal Convolutiva

Vamos de regreso a la capa max pooleada y vamos a hacernos un poco de espacio. En términos estrictos ya terminamos nuestra red neuronal convolutiva – pero todavía no tenemos un resultado.

Todas las redes neuronales convolucionales – las que te enseñan en la escuela al menos, siempre terminan en una red neuronal conectada, densa, tradicional, de las que vimos en los capítulos anteriores, o como quieras verlo.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1



1	0
0	1

3.33 Funcionamiento de una Red Neuronal Convolutiva

Vamos a convertir nuestra capa Max Pooleada en una columna de nodos de entrada (ósea una capa flatten si gustas), y esos nodos los meteremos en una red neuronal común y corriente.

Esta red neuronal tiene 4 nodos de entrada y una sola capa escondida, con un solo nodo usando la función de activación RELU y 2 neuronas de salida – una para el 0 y otra para el 1.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1



1	0
0	1

Flatten()

2x2 --> 4

3x3 -> 9

3.34 Funcionamiento de una Red Neuronal Convolutiva

Vamos a convertir nuestra capa Max Pooleada en una columna de nodos de entrada (ósea una capa flatten si gustas), y esos nodos los meteremos en una red neuronal común y corriente.

Esta red neuronal tiene 4 nodos de entrada y una sola capa escondida, con un solo nodo usando la función de activación RELU y 2 neuronas de salida – una para el 0 y otra para el 1.

1	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	1



1	0
0	1

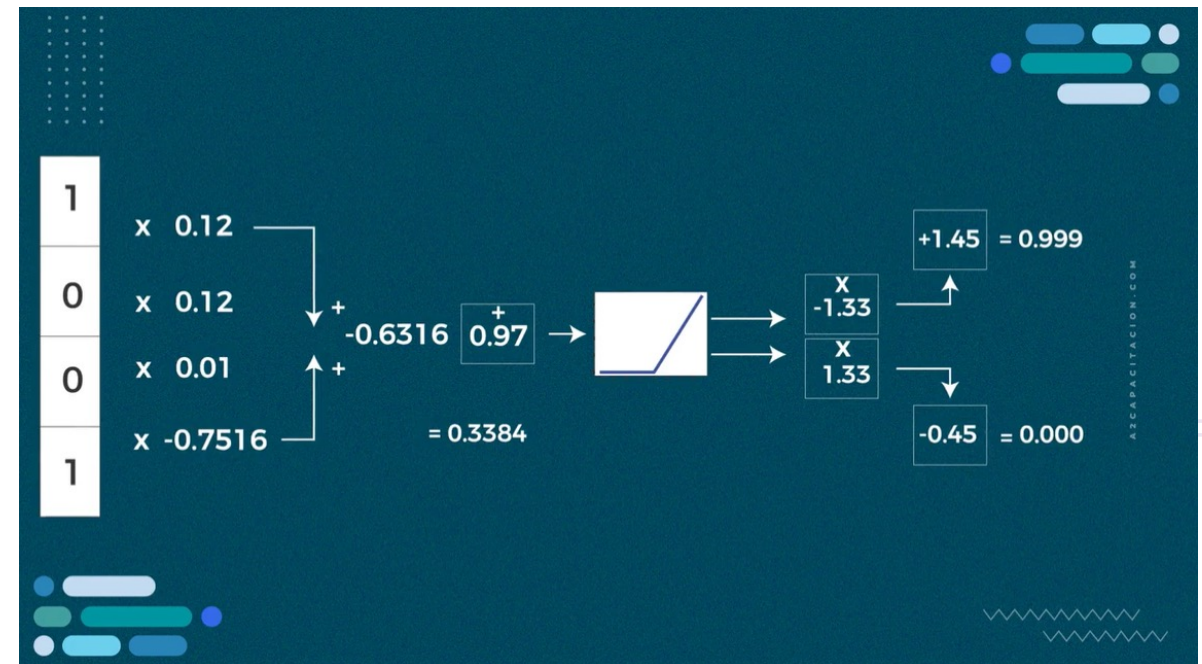
3.35 Funcionamiento de una Red Neuronal Convolutiva

Le damos nuestra imagen del 0, la pasamos por el filtro para crear el feature map, luego pasamos el feature map por la función de activación RELU, luego aplicamos max pooling y sacamos los valores máximos, y esos valores máximos los metemos en nuestras neuronas de entrada y acabamos con estos valores.

Ahora, multiplicamos los valores en las neuronas de entrada por sus pesos asociados y sumamos cada termino y obtenemos el bias y nos da este resultado de aquí.

Así que pasamos este valor por RELU y claro que como es positivo nos devuelve el mismo valor.

Ahora esta conexión de RELU a la salida para el número 0 no da un valor de 1 y la otra conexión para el 1 nos da un valor de 0.



Preguntas

